

PREDICTING CYBERBULLYING ON SOCIAL MEDIA IN THE BIG DATA ERA USING MACHINE LEARNING ALGORITHMS

A Project Report

Submitted in the partial fulfilment of the requirements for the award of

The degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by

S. SIVA MEGHANA

(228A1A05A1)

Under the Esteemed Guidance of

Dr. K. PURNA CHANDRA RAO

Asso.Prof



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

RISE KRISHNA SAI PRAKASAM GROUP OF INSTITUTIONS

(Affiliated to JNTU, Kakinada & Approved by AICTE, New Delhi)

Accredited by NBA and NAAC "A" certified by ISO 9001:2015

Valluru-523272



RISE KRISHNA SAI PRAKASAM GROUP OF INSTITUTIONS

(Affiliated to JNTU, Kakinada & Approved by AICTE, New Delhi) Accredited
by NBA, NAAC with 'A' certified by ISO 9001:2015,
Valluru-523272

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(2022-2026)

CERTIFICATE

This is to certify that this project work entitled “**PREDICTING CYBERBULLYING ON SOCIAL MEDIA IN THE BIG DATA ERA USING MACHINE LEARNING ALGORITHMS**” is being submitted by S. SIVA MEGHANA (228A1A05A1) in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in COMPUTER SCIENCE & ENGINEERING from JNTUK, Kakinada during the academic year 2025- 2026 is a record of Bonafide work carried out by me under my esteemed guidance and supervision.

PROJECT GUIDE

Dr. K. Purna Chandra Rao

Asso.Prof

HEAD OF THE DEPARTMENT

Dr. G. Kishore

Professor & HOD

EXTERNAL EXAMINER

DECLARATION

I hereby declare that the work is being presented in this dissertation entitled **“PREDICTING CYBERBULLYING ON SOCIAL MEDIA IN THE BIGDATA ERA USING MACHINE LEARNING ALGORITHMS”** submitted towards the partial fulfillment of requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering, work carried out under the supervision of **Dr. K. PURNA CHANDRA RAO**, Associate Professor, **RISE KRISHNA SAI PRAKASAM GROUP OF INSTITUTIONS, Valluru**. The results embodied in this dissertation report have not been submitted to any other University for the award of any other degree. Furthermore, the technical details furnished in various chapters of this report are purely relevant to the above project and there is no deviation from the theoretical point of view for design, development, and implementation.

S. SIVA MEGHANA
(228A1A05A1)

ACKNOWLEDGEMENT

I would like to express our sincere gratitude to our guide **Dr. K. Purna Chandra Rao**, Associate Professor, for providing his valuable guidance, comments, suggestions and support throughout the course of the project.

I express our heartfelt thanks to **Dr. G. Kishore**, Head of the Department, Computer Science and Engineering, for his valuable support to bring out this project in time.

I pay our profound sense of gratitude to our Principal **Prof. Dr A.V. Bhaskara Rao** for providing an excellent environment in our college and helping us at all points for achieving our task.

I would like to express our sincere thanks to the **Management of RISE KRISHNA SAI PRAKASAM GROUP OF INSTITUTIONS, Valluru** for providing a good environment and infrastructure.

Finally, I thank all our faculty members, supporting **Staff** of CSE Department and friends for their kind co-operation and valuable help for the completion of the project.

S. SIVA MEGHANA
(228A1A05A1)



RISE KRISHNA SAI PRAKASAM GROUP OF INSTITUTIONS

(Affiliated to JNTU, Kakinada & Approved by AICTE, New Delhi)

Accredited by NBA, NAAC with 'A' certified by ISO 9001:2015

Valluru-523272

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Visions of the Institute	To be a premier institution in technical education by creating professionals of global standards with ethics and social responsibility for the development of the nation and the mankind.
Mission of the Institute	Impart Outcome Based Education through well qualified and dedicated faculty.
	Provide state-of-the-art infrastructure and facilities for application-oriented research.
	Reinforce technical skills with life skills and entrepreneurship skills.
	Promote cutting-edge technologies to produce industry-ready Professionals.
	Facilitate interaction with all stakeholders to foster ideas and innovation.
	Inculcate moral values, professional ethics, and social responsibility
Vision of the Department	To be a center of excellence in computer science and engineering for value- based education to serve humanity and contribute for socio- economic development.
Mission of the Department	Provide professional knowledge by student centric teaching-learning process to contribute software industry.
	Inculcate training on cutting edge technologies for industry needs.
	Create an academic ambiance leading to research.
	Promote industry institute interaction for real time problem solving.

Program Educational Objectives (PEOs)

Graduates of the Program will be able to

PEO1	Develop software solutions for real world problems by applying Mathematics, Science and engineering principles.
PEO2	Function as members of multi-disciplinary teams and to communicate effectively using modern tools.
PEO3	Pursue career in software industry or higher studies with continuous learning and apply software knowledge.
PEO4	Practice the profession with ethics, integrity, leadership and social responsibility.

Program Outcomes (POs)

PO1	Engineering Knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
PO2	Problem Analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
PO3	Design/Development of Solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
PO4	Conduct Investigations of Complex Problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
PO5	Modern Tool Usage: Create, Select and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6	The Engineer and Society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent Responsibilities relevant to the professional engineering practice.
-----	--

PO7	Environmental and Sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
PO8	Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
PO9	Individual and Team Work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
PO10	Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
PO11	Project Management and Finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
PO12	Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Program Specific Outcomes (PSOs):

A student of the Computer Science and Engineering Program will be able to

PSO1	Domain Knowledge: Apply the Knowledge of Programming Languages, Networks and Databases for design and development of Software Applications.
PSO2	Computing Paradigms: Understand the evolutionary changes in computing possess knowledge of context aware responsibility of paradigms and meet the challenges of the future.



RISE KRISHNA SAI PRAKASAM GROUP OF INSTITUTIONS

(Affiliated to JNTU, Kakinada & Approved by AICTE, New Delhi)

Accredited by NBA, NAAC with 'A' certified by ISO 9001:2015

Vallur-523272

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Name of the Course: Major Project

Year & Semester: IV & II

Academic Year : 2025-2026

Regulation : R20

Course Outcomes:

Course Outcomes (COs)		Taxonomy Level
C421.1	Develop application for community needs	Applying
C421.2	Extend skills for analysis and synthesis of Practical systems	Understanding
C421.3	Get hold of the use of new tools efficiently and creatively.	Analyzing
C421.4	Work in team to carry out analysis and cost-effective, environmentally friendly designs of engineering systems.	Analyzing
C421.5	Write Technical/Project reports and oral presentation of the work done to an audience.	Applying
C421.6	Demonstrate a product developed.	Understanding

Project Coordinator

Head of the department

CO Vs PO Mapping

Course Outcomes (CO's)	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12
C421.1	2	2	3	3	3	3	2	2	3	2	2	2
C421.2	2	2	3	3	3	3	2	2	3	3	3	3
C421.3	2	2	3	3	3	3	2	2	3	2	2	2
C421.4	2	2	3	3	3	3	3	3	3	3	3	3
C421.5	2	2	3	2	3	3	2	3	3	3	3	3
C421.6	2	2	2	2	3	3	3	3	3	3	3	3
C424	2.00	2.00	2.83	2.67	3.00	3.00	2.33	2.50	3.00	2.67	2.67	2.67

CO Vs PSO Mapping

Course Outcomes (Cos)	PSO 1	PSO 2
C421.1	3	3
C421.2	3	3
C421.3	3	3
C421.4	3	3
C421.5	2	2
C421.6	3	3
	2.83	2.83

1: Low

2: Medium

3: High

Guide Signature

**PREDICTING CYBERBULLYING ON SOCIAL
MEDIA IN THE BIG DATA
ERA USING MACHINE LEARNING
ALGORITHMS**

INDEX

S.NO	CONTENTS	Page No
	ABSTRACT	1
1.	INTRODUCTION	2-4
	1.1 Introduction	
	1.2 Objective	
	1.3 Problem Definition	
2.	LITERATURE SURVEY	5-7
3.	SYSTEM ANALYSIS	8-11
	3.1 Existing System	
	3.1.1 Disadvantages of Existing System	
	3.2 Proposed System	
	3.2.1 Advantages of Proposed System	
	3.3 Functional Requirements	
	3.4 Non-Functional Requirements	
4.	FEASIBILITY STUDY	12-14
5.	SYSTEM REQUIREMENTS	15-16
	a. Software Requirements	

b. Hardware Requirements

6.	SYSTEM ENVIRONMENT	17-42
7.	SYSTEM DESIGN	43-55
	7.1 System Architecture	
	7.2 UML Diagrams	
8.	IMPLEMENTATION	56-69
	Module	
	Source Code	
9.	SYSTEM TESTING	70-75
10.	SCREENSHOTS	76-81
11.	CONCLUSION& FUTURE ENHANCEMENT	82-83
12.	BIBLIOGRAPHY	84-86

LIST OF FIGURES

S.NO	NAME OF FIGURE	PAGE NO
1.	SCREENSHOT 1	37
2.	SCREENSHOT 2	37
3.	SCREENSHOT 3	38
4.	SCREENSHOT 4	38
5.	SCREENSHOT 5	39
6.	SCREENSHOT 6	40
7.	SCREENSHOT 7	40
8.	SCREENSHOT 8	41
9.	SCREENSHOT 9	41
10.	SCREENSHOT 10	42
11.	SYSTEM ARCHITECTURE	44
12.	FLOWCHART	45-46
13.	DATAFLOW DIAGRAM	47
14.	USE CASE DIAGRAM	49
15.	CLASS DIAGRAM	50
16.	OBJECT DIAGRAM	51
17.	STATE DIAGRAM	52
18.	ACTIVITY DIAGRAM	53

19.	SEQUENCE DIAGRAM	54
20.	COLLABORATION DIAGRAM	55
21.	SCREENSHOT 11	77
22.	SCREENSHOT 12	77
23.	SCREENSHOT 13	78
24.	SCREENSHOT 14	78
25.	SCREENSHOT 15	79
26.	SCREENSHOT 16	79
27.	SCREENSHOT 17	80
28.	SCREENSHOT 18	80
29.	SCREENSHOT 19	81
30.	SCREENSHOT 20	81

S.NO	TABLE	PAGENO
1	TEST CASES	72-75

ABSTRACT

With the rapid growth of Information and Communication Technologies (ICT), social interactions have expanded far beyond traditional geographical and cultural boundaries. Modern communication platforms, especially social media, have enabled large-scale user interaction, resulting in the generation of vast amounts of behavioural data. While these platforms offer numerous benefits, they have also introduced new challenges, particularly the rise of aggressive and harmful behaviour in online environments.

This project focuses on identifying and analysing cyberbullying behaviour on social media platforms. It explores how aggressive interactions manifest in online communication and aims to develop an effective prediction system to detect such behaviour. The work involves collecting relevant data, processing it, and extracting meaningful features that can represent user behaviour patterns.

Special emphasis is placed on selecting the most significant features that contribute to accurate prediction. Various machine learning algorithms are applied and evaluated to classify and predict cyberbullying activities. The performance of these models is analysed to determine the most effective approach.

Additionally, this project discusses the challenges encountered during the development process and highlights potential improvements. The outcomes of this work provide a foundation for building more reliable and efficient systems to detect and prevent cyberbullying in online platforms.

CHAPTER 1

INTRODUCTION

1.INTRODUCTION

1.1 INTRODUCTION

Machine or deep learning algorithms help researchers understand big data. Abundant information on humans and their societies can be obtained in this big data era, but this acquisition was previously impossible. One of the main sources of human-related data is social media (SM). By applying machine learning algorithms to SM data, we can exploit historical data to predict the future of a wide range of applications. Machine learning algorithms provide an opportunity to effectively predict and detect negative forms of human behavior, such as cyberbullying. Big data analysis can uncover hidden knowledge through deep learning from raw data. Big data analytics has improved several applications, and forecasting the future has even become possible through the combination of big data and machine learning algorithms.

An insightful analysis of data on human behavior and interaction to detect and restrain aggressive behavior involves multifaceted angles and aspects and the merging of theorems and techniques from multidisciplinary and interdisciplinary fields. The accessibility of large-scale data produces new research questions, novel computational methods, interdisciplinary approaches, and outstanding opportunities to discover several vital inquiries quantitatively. However, using traditional methods (statistical methods) in this context is challenging in terms of scale and accuracy. These methods are commonly based on organized data on human behavior and small-scale human networks (traditional social networks). Applying these methods to large online social networks (OSNs) in terms of scale and extent causes several issues. On the one hand, the explosive growth of OSNs enhances and disseminates aggressive forms of behavior by providing platforms and networks to commit and propagate such behavior. On the other hand, OSNs offer important data for exploring human behavior and interaction at a large scale, and these data can be used by researchers to develop effective methods of detecting and restraining misbehavior and/or aggressive behavior. OSNs provide criminals with tools to perform aggressive actions and networks to commit misconduct. Therefore, methods that address both aspects (content and network) should be optimized to detect and restrain aggressive behavior in complex systems.

1.2 OBJECTIVE

The main objective of this project is to develop an intelligent and automated system capable of detecting cyberbullying content on social media platforms by analyzing large-scale data. It aims

to handle vast amounts of unstructured textual data generated from online interactions using Big Data technologies. The project focuses on collecting relevant datasets, preprocessing the data to remove noise and irrelevant information, and transforming the text into meaningful numerical features. It further seeks to apply machine learning algorithms to accurately classify content as bullying or non-bullying. Another important objective is to ensure scalability and efficiency in data processing using frameworks such as Apache Spark. The system is designed to minimize manual monitoring efforts while improving detection accuracy through proper evaluation metrics like precision, recall, and F1-score. Additionally, the project aims to create a foundation that can be extended to real-time applications, ultimately contributing to a safer and more secure online environment.

1.3 PROBLEM DEFINITION

The rapid growth of social media platforms has led to an enormous increase in user-generated content, where harmful interactions such as cyberbullying have become a serious concern. Cyberbullying involves the use of offensive, abusive, or threatening language that can negatively impact individuals' mental and emotional well-being. Due to the massive volume, high velocity, and unstructured nature of social media data, it is extremely difficult to manually monitor and identify such harmful content in real time. Traditional methods are inefficient and unable to scale with the continuously growing data. Therefore, there is a need for an automated and scalable system that can efficiently process large amounts of data and accurately detect cyberbullying content. This project addresses the challenge by proposing a Big Data-based approach that uses data processing frameworks and machine learning techniques to identify and classify cyberbullying in social media environments.

CHAPTER 2

LITERATURE SURVEY

2. LITERATURE SURVEY

2.1 Predicting Human Behavior Using Computational Models

AUTHORS: V. Subrahmanian and S. Kumar

ABSTRACT: This work explores computational techniques for predicting human behavior using large-scale data sources such as social media, mobile sensors, and digital footprints. The authors discuss probabilistic models, machine learning frameworks, and network-based inference methods to forecast actions ranging from consumer behavior to criminal activity. The paper highlights emerging challenges including data sparsity, privacy preservation, ethical implications, and adversarial manipulation, positioning behavioral prediction as a key future research frontier.

2.2 Homophily in Online Social Networks

AUTHORS: H. Lauw, J. C. Shafer, R. Agrawal, and A. Ntoulas

ABSTRACT: This study investigates homophily—the tendency of individuals with similar characteristics to form connections—in the LiveJournal social network. Using statistical and graph-based analysis, the authors demonstrate how user interests and behaviors strongly influence network formation. The results provide insight into recommendation systems, community detection, and influence propagation, emphasizing the structural patterns underlying online social interactions.

2.3 Cybercrime and Cyberbullying Detection on Twitter

AUTHORS: M. A. Al-Garadi, K. D. Varathan, and S. D. Ravana

ABSTRACT: The paper presents an experimental evaluation of cybercrime detection techniques, specifically targeting cyberbullying within Twitter communications. Various machine learning classifiers and linguistic feature extraction approaches are employed to analyze message content and user interaction patterns. Performance comparisons reveal that supervised learning models can effectively identify abusive behavior, highlighting the importance of contextual and semantic features in online threat detection.

2.4 Social Media–Based Future Prediction: A Systematic Review

AUTHORS: L. Phillips, C. Dowling, K. Shaffer, N. Hodas, and S. Volkova

ABSTRACT: This systematic literature review surveys how social media data is utilized for predicting real-world events such as political trends, disasters, disease outbreaks, and financial movements. The authors categorize methodologies into natural language processing, network analysis, and temporal modeling. The study identifies limitations related to dataset bias, evaluation inconsistency, and reproducibility, and outlines research opportunities for robust and explainable predictive analytics.

2.5 Technical Survey of Online Social Networks and Services

AUTHORS: H. Quan, J. Wu, and Y. Shi

ABSTRACT: This survey provides a technical overview of online social networks and social networking services, covering system architectures, communication protocols, privacy mechanisms, and data management strategies. The authors analyze challenges associated with scalability, security threats, and user trust, and discuss how evolving platforms influence information diffusion and digital communities. The work serves as a foundational reference for research in social network infrastructure and analytics.

CHAPTER 3

SYSTEM ANALYSIS

3. SYSTEM ANALYSIS

3.1 Existing System

- ❖ State-of-the-art research has developed features to improve the performance of cyberbullying prediction. For example, a lexical syntactic feature has been proposed to deal with the prediction of offensive language; this method is better than traditional learning-based approaches in terms of precision. Dadvar *et al.* examined gender information from profile information and developed a gender-based approach for cyberbullying prediction by using datasets from Myspace as a basis. The gender feature was selected to improve the discrimination capability of a classifier. Age and gender were included as features in other studies, but these features are limited to the information provided by users in their online profiles.
- ❖ Several studies focused on cyberbullying prediction based on profane words as a feature. Similarly, a lexicon of profane words was constructed to indicate bullying, and these words were used as features for input to machine learning algorithms. Using profane words as features demonstrates a significant improvement in model performance. For example, the number of ``bad" words and the density of ``bad" words were proposed as features for input to machine learning in a previous work. The study concluded that the percentage of ``bad" words in a text is indicative of cyberbullying. Another research expanded a list of pre-defined profane words and allocated different weights to create bullying features. These features were concatenated with bag-of- words and latent semantic features and used as a feature input for a machine learning algorithm.

3.1.1 DISADVANTAGES OF EXISTING SYSTEM:

- The System is not much affective due to Semi supervised machine learning techniques.
- The system doesn't have sentiment classification for cyberbullying.

3.2 Proposed System

- The proposed system is constructing cyberbullying prediction models is to use a text classification approach that involves the construction of machine learning classifiers from labeled text instances. Another means is to use a lexicon-based model that involves computing orientation for a document from the semantic orientation of words or phrases in the document. Generally, the lexicon in lexicon-based models can be constructed manually or automatically

by using seed words to expand the list of words. However, cyberbullying prediction using the lexicon-based approach is rare in literature.

- The primary reason is that the texts on SM websites are written in an unstructured manner, thus making it difficult for the lexicon-based approach to detect cyberbullying based only on lexicons. However, lexicons are used to extract features, which are often utilized as inputs to machine learning algorithms. For example, lexicon-based approaches, such as using a profane-based dictionary to detect the number of profane words in a post, are adopted as profane features to machine learning models. The key to effective cyberbullying prediction is to have a set of features that are extracted and engineered

3.2.1 ADVANTAGES OF PROPOSED SYSTEM:

- The system is more effective due to LOGISTIC REGRESSION CLASSIFICATION and UNSUPERVISED MACHINE LEARNING.
- An effective cyberbullying prediction models is to use a text classification approach that involves the construction of machine learning classifiers from labeled text instance and also is to use a lexicon-based model that involves computing orientation for a document from the semantic orientation of words or phrases in the document.

3.3 Functional Requirements

1. Data Collection
2. Data Preprocessing
3. Training And Testing
4. Modeling
5. Predicting

3.4 Non-Functional Requirements

NON-FUNCTIONAL REQUIREMENT (NFR) specifies the quality attribute of a software system. They judge the software system based on Responsiveness, Usability, Security, Portability and other non-functional standards that are critical to the success of the software system. Example of nonfunctional requirement, *“how fast does the website load?”* Failing to meet non-functional requirements can result in systems that fail to satisfy user needs. Non-functional Requirements

allows you to impose constraints or restrictions on the design of the system across the various agile backlogs. Example, the site should load in 3 seconds when the number of simultaneous users are > 10000 . Description of non-functional requirements is just as critical as a functional requirement.

Usability requirement

Serviceability requirement

Manageability requirement

Recoverability requirement

Security requirement

Data Integrity requirement

Capacity requirement

Availability requirement

Scalability requirement

Interoperability requirement

Reliability requirement

Maintainability requirement

Regulatory requirement

Environmental requirement

CHAPTER 4

FEASIBILITY STUDY

4. FEASIBILITY STUDY

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

Three key considerations involved in the feasibility analysis are

- ◆ **ECONOMICAL FEASIBILITY**
- ◆ **TECHNICAL FEASIBILITY**
- ◆ **SOCIAL FEASIBILITY**

ECONOMICAL FEASIBILITY

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus, the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

TECHNICAL FEASIBILITY

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

SOCIAL FEASIBILITY

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system.

CHAPTER 5

SYSTEM REQUIREMENTS

5. SYSTEM REQUIREMENTS

5.1 SOFTWARE REQUIREMENTS

The functional requirements or the overall description documents include the product perspective and features, operating system and operating environment, graphics requirements, design constraints and user documentation.

The appropriation of requirements and implementation constraints gives the general overview of the project in regards to what the areas of strength and deficit are and how to tackle them.

- **Python idle 3.8 version (or)**
- **Anaconda 3.7 (or)**
- **Jupyter (or)**
- **Google Collab**

5.2 HARDWARE REQUIREMENTS

Minimum hardware requirements are very dependent on the particular software being developed by a given Enthought Python / Canopy / VS Code user. Applications that need to store large arrays/objects in memory will require more RAM, whereas applications that need to perform numerous calculations or tasks more quickly will require a faster processor.

- **Operating system** : windows, Linux
- **Processor** : minimum intel i3
- **Ram** : minimum 4 GB
- **Hard disk** : minimum 250 GB

CHAPTER 6

SYSTEMENVIRONMENT

6. SYSTEM ENVIRONMENT

What is Python

Below are some facts about Python.

Python is currently the most widely used multi-purpose, high-level programming language.

Python allows programming in Object-Oriented and Procedural paradigms. Python programs generally are smaller than other programming languages like Java.

Programmers have to type relatively less and indentation requirement of the language, makes them readable all the time.

Python language is being used by almost all tech-giant companies like – Google, Amazon, Facebook, Instagram, Dropbox, Uber... etc.

The biggest strength of Python is huge collection of standard libraries which can be used for the following –

Machine Learning

- GUI Applications (like Kivy, Tkinter, PyQt etc.)
- Web frameworks like Django (used by YouTube, Instagram, Dropbox)
- Image processing (like Open cv, Pillow)
- Web scraping (like Scrapy, BeautifulSoup, Selenium)
- Test frameworks
- Multimedia

Advantages of Python

Let us see how Python dominates over other languages.

1. Extensive Libraries

Python downloads with an extensive library and it contains code for various purposes like regular expressions, documentation-generation, unit-testing, web browsers, threading, databases, CGI, email, image manipulation, and more. So, we do not have to write the complete code for that manually.

2. Extensible

As we have seen earlier, Python can be **extended to other languages**. You can write some of your code in languages like C++ or C. This comes in handy, especially in projects.

3. Embeddable

Complimentary to extensibility, Python is embeddable as well. You can put your Python code in your source code of a different language, like C++. This lets us add **scripting capabilities** to our code in the other language.

4. Improved Productivity

The language's simplicity and extensive libraries render programmers **more productive** than languages like Java and C++ do. Also, the fact that you need to write less and get more things done.

5. IOT Opportunities

Since Python forms the basis of new platforms like Raspberry Pi, it finds the future bright for the Internet of Things. This is a way to connect the language with the real world.

6. Simple and Easy

When working with Java, you may have to create a class to print '**Hello World**'. But in Python, just a print statement will do. It is also quite **easy to learn, understand, and code**. This is why when people pick up Python, they have a hard time adjusting to other more verbose languages like Java.

7. Readable

Because it is not such a verbose language, reading Python is much like reading English. This is the reason why it is so easy to learn, understand, and code. It also does not need curly braces to define blocks, and **indentation is mandatory**. These further aids the readability of the code.

8. Object-Oriented

This language supports both the **procedural and object-oriented** programming paradigms. While functions help us with code reusability, classes and objects let us model the real world. A class allows the encapsulation of data and functions into one.

9. Free and Open-Source

Like we said earlier, Python is **freely available**. But not only can you **download Python** for free, but you can also download its source code, make changes to it, and even distribute it. It downloads with an extensive collection of libraries to help you with your tasks.

10. Portable

When you code your project in a language like C++, you may need to make some changes to it if you want to run it on another platform. But it isn't the same with Python. Here, you need to **code only once**, and you can run it anywhere. This is called **Write Once Run Anywhere (WORA)**. However, you need to be careful enough not to include any system-dependent features.

11. Interpreted

Lastly, we will say that it is an interpreted language. Since statements are executed one by one, **debugging is easier** than in compiled languages.

Advantages of Python Over Other Languages

1. Less Coding

Almost all of the tasks done in Python require less coding when the same task is done in other languages. Python also has an awesome standard library support, so you do not have to search for any third-party libraries to get your job done. This is the reason that many people suggest learning Python to beginners.

2. Affordable

Python is free therefore individuals, small companies or big organizations can leverage the free available resources to build applications. Python is popular and widely used so it gives you better community support. The 2019 GitHub annual survey showed us that Python has overtaken Java in the most popular programming language category.

3. Python is for Everyone

Python code can run on any machine whether it is Linux, Mac or Windows. Programmers need to learn different languages for different jobs but with Python, you can professionally build web apps, perform data analysis and machine learning, automate things, do web scraping and build games and powerful visualizations. It is an all- round programming language

Disadvantages of Python

So far, we have seen why Python is a great choice for your project. But if you choose it, you should be aware of its consequences as well. Let us now see the downsides of choosing Python over another language.

1. Speed Limitations

We have seen that Python code is executed line by line. But since Python is interpreted, it often results in slow execution. This, however, is not a problem unless speed is a focal point for the project. In other words, unless high speed is a requirement, the benefits offered by Python are enough to distract us from its speed limitations.

2. Weak in Mobile Computing and Browsers

While it serves as an excellent server-side language, Python is rarely seen on the **client-side**. Besides that, it is rarely ever used to implement smartphone-based applications. One such application is called **Carbonells**. The reason it is not so famous despite the existence of python is that it isn't that secure.

3. Design Restrictions

As you know, Python is **dynamically typed**. This means that you don't need to declare the type of variable while writing the code. It uses **duck-typing**. But wait, what is that? Well, it just means that if it looks like a duck, it must be a duck. While this is easy on the programmers during coding, it can **raise run-time errors**.

4. Underdeveloped Database Access Layers

Compared to more widely used technologies like **JDBC (Java Database Connectivity)** and **ODBC (Open Database Connectivity)**, Python's database access layers are a bit underdeveloped. Consequently, it is less often applied in huge enterprises.

5. Simple

No, we are not kidding. Python's simplicity can indeed be a problem. Take my example. I do not do Java, I am more of a Python person. To me, its syntax is so simple that the verbosity of Java code seems unnecessary. This was all about the Advantages and Disadvantages of Python Programming Language.

History of Python

What do the alphabet and the programming language Python have in common? Right, both start with ABC. If we are talking about ABC in the Python context, it's clear that the programming language ABC is meant. ABC is a general-purpose programming language and programming environment, which had been developed in the Netherlands, Amsterdam, at the CWI (Centrum Wiskunde & Informatica). The greatest achievement of ABC was to influence the design of Python. Python was conceptualized in the late 1980s. Guido van Rossum worked at that time on a project at the CWI, called Amoeba, a distributed operating system. In an interview with Bill Venners, Guido van Rossum said: "In the early 1980s, I worked as an implementer on a team building a language called ABC at Centrum voor Wiskunde en Informatica (CWI). I don't know how well people know ABC's influence on Python. I try to mention ABC's influence because I'm indebted to everything I learned during that project and to the people who worked on it." Later in the same Interview, Guido van Rossum continued: "I remembered all my experience and some of my frustration with ABC. I decided to try to design a simple scripting language that possessed some of ABC's better properties, but without its problems. So, I started typing. I created a simple virtual machine, a simple parser, and a simple runtime. I made my own version of the various ABC parts that I liked. I created a basic syntax, used indentation for statement grouping instead of curly braces or begin-end blocks, and developed a small number of powerful data types: a hash table (or dictionary, as we call it), a list, strings, and numbers."

What is Machine Learning?

Machine Learning is a system of computer algorithms that can learn from example through self- improvement without being explicitly coded by a programmer. Machine learning is a part of artificial Intelligence which combines data with statistical tools to predict an output which can be used to make actionable insights.

The breakthrough comes with the idea that a machine can singularly learn from the data (i.e., example) to produce accurate results. Machine learning is closely related to data mining and Bayesian predictive modeling. The machine receives data as input and uses an algorithm to formulate answers.

A typical machine learning task is to provide a recommendation. For those who have a Netflix account, all recommendations of movies or series are based on the user's historical data. Tech companies are using unsupervised learning to improve the user experience with personalizing recommendations.

Machine learning is also used for a variety of tasks like fraud detection, predictive maintenance, portfolio optimization, automatize task and so on.

Need for Machine Learning

Human beings, at this moment, are the most intelligent and advanced species on earth because they can think, evaluate and solve complex problems. On the other hand, AI is still in its initial stage and has not surpassed human intelligence in many aspects. Then the question is what is the need to make machine learn? The most suitable reason for doing this is, “to make decisions, based on data, with efficiency and scale.”

Lately, organizations are investing heavily in newer technologies like Artificial Intelligence, Machine Learning and Deep Learning to get the key information from data to perform several real- world tasks and solve problems. We can call it data-driven decisions taken by machines, particularly to automate the process. These data-driven decisions can be used, instead of using programming logic, in problems that cannot be programmed inherently.

Challenges in Machines Learning

While Machine Learning is rapidly evolving, making significant strides with cyber security and autonomous cars, this segment of AI as whole still has a long way to go. The reason behind is that ML has not been able to overcome several challenges. The challenges that ML is facing currently are:

Quality of data – Having good-quality data for ML algorithms is one of the biggest challenges. Use of low-quality data leads to problems related to data preprocessing and feature extraction.

Time-Consuming task – Another challenge faced by ML models is the consumption of time especially for data acquisition, feature extraction and retrieval.

Lack of specialist persons – As ML technology is still in its infancy stage, availability of expert resources is a tough job.

No clear objective for formulating business problems – Having no clear objective and well-defined goal for business problems is another key challenge for ML because this technology is not that mature yet.

Issue of overfitting & underfitting – If the model is overfitting or underfitting, it cannot be represented well for the problem.

Curse of dimensionality – Another challenge ML model faces is too many features of datapoints. This can be a real hindrance.

Difficulty in deployment – Complexity of the ML model makes it quite difficult to be deployed in real life.

Applications of Machines Learning

Machine Learning is the most rapidly growing technology and according to researchers we are in the golden year of AI and ML. It is used to solve many real-world complex problems which cannot be solved with traditional approach. Following are some real-world applications of ML:

- Emotion analysis
- Sentiment analysis
- Error detection and prevention

Weather forecasting and prediction Before we take a look at the details of various machine learning methods, let's start by looking at what machine learning is, and what it isn't. Machine learning is often categorized as a subfield of artificial intelligence, but I find that categorization can often be misleading at first brush. The study of machine learning certainly arose from research in this context, but in the data science application of machine learning methods, it's more helpful to think of machine learning as a means of *building models of data*.

Fundamentally, machine learning involves building mathematical models to help understand data. "Learning" enters the fray when we give these models *tunable parameters* that can be adapted to observed data; in this way the program can be considered to be "learning" from the data. Once these models have been fit to previously seen data, they can be used to predict and understand aspects of newly observed data. I'll leave to the reader the more philosophical digression regarding the extent to which this type of mathematical, model-based "learning" is similar to the "learning" exhibited by the human brain. Understanding the problem setting in machine learning is essential to using these tools effectively, and so we will start with some broad categorizations of the types of approaches we'll discuss here.

Categories Of Machine Learning: -

At the most fundamental level, machine learning can be categorized into two main types: supervised learning and unsupervised learning.

Supervised learning involves somehow modeling the relationship between measured features of data and some label associated with the data; once this model is determined, it can be used to apply labels to new, unknown data. This is further subdivided into *classification* tasks and *regression* tasks: in classification, the labels are discrete categories, while in regression, the labels are continuous quantities. We will see examples of both types of supervised learning in the following section.

Unsupervised learning involves modeling the features of a dataset without reference to any label, and is often described as "letting the dataset speak for itself." These models include tasks such as *clustering* and *dimensionality reduction*. Clustering algorithms identify distinct groups of data, while dimensionality reduction algorithms search for more succinct representations of the data. We will see examples of both types of unsupervised learning in the following section.

Need for Machine Learning

Human beings, at this moment, are the most intelligent and advanced species on earth because they can think, evaluate and solve complex problems. On the other side, AI is still in its initial stage and haven't surpassed human intelligence in many aspects. Then the question is that what is the need to make machine learn? The most suitable reason for doing this is, "to make decisions, based on data, with efficiency and scale".

Lately, organizations are investing heavily in newer technologies like Artificial Intelligence, Machine Learning and Deep Learning to get the key information from data to perform several real-world tasks and solve problems. We can call it data-driven decisions taken by machines, particularly to automate the process. These data-driven decisions can be used, instead of using programming logic, in the problems that cannot be programmed inherently. The fact is that we can't do without human intelligence, but other aspect is that we all need to solve real-world problems with efficiency at a huge scale. That is why the need for machine learning arises.

Challenges in Machines Learning: -

While Machine Learning is rapidly evolving, making significant strides with cybersecurity and autonomous cars, this segment of AI as whole still has a long way to go. The reason behind is that ML has not been able to overcome number of challenges. The challenges that ML is facing currently are –

Quality of data – Having good-quality data for ML algorithms is one of the biggest challenges. Use of low- quality data leads to the problems related to data preprocessing and feature extraction.

Time-Consuming task – Another challenge faced by ML models is the consumption of time especially for data acquisition, feature extraction and retrieval.

Lack of specialist persons – As ML technology is still in its infancy stage, availability of expert resources is a tough job.

No clear objective for formulating business problems – Having no clear objective and well-defined goal for business problems is another key challenge for ML because this technology is not that mature yet.

Issue of overfitting & underfitting – If the model is overfitting or underfitting, it cannot be represented well for the problem

Curse of dimensionality – Another challenge ML model faces is too many features of data points. This can be a real hindrance.

Difficulty in deployment – Complexity of the ML model makes it quite difficult to be deployed in real life.

Applications of Machines Learning: -

Machine Learning is the most rapidly growing technology and according to researchers we are in the golden year of AI and ML. It is used to solve many real-world complex problems which cannot be solved with traditional approach. Following are some real-world applications of ML

- Emotion analysis
- Sentiment analysis
- Error detection and prevention
- Weather forecasting and prediction
- Stock market analysis and forecasting
- Speech synthesis
- Speech recognition
- Customer segmentation
- Object recognition
- Fraud detection
- Fraud prevention
- Recommendation of products to customer in online shopping

How to Start Learning Machine Learning?

Arthur Samuel coined the term “**Machine Learning**” in 1959 and defined it as a “**Field of study that gives computers the capability to learn without being explicitly programmed**” and that was the beginning of Machine Learning! In modern times, Machine Learning is one of the most popular (if not the most!) career choices. According to indeed, Machine Learning Engineer is the Best Job of 2019 with a 344% growth and an average base salary of **\$146,085** per year.

But there is still a lot of doubt about what exactly Machine Learning is and how to start learning it. So, this article deals with the Basics of Machine Learning and the path you can follow to eventually become a full- fledged Machine Learning Engineer. Now let’s get started!!!

How to start learning ML?

This is a rough roadmap you can follow on your way to becoming an insanely talented Machine Learning Engineer. Of course, you can always modify the steps according to your needs to reach your desired end-goal!

Step 1 – Understand the Prerequisites

In case you are a genius, you could start ML directly but normally, there are some prerequisites that you need to know which include Linear Algebra, Multivariate Calculus, Statistics, and Python. And if you don’t know these, never fear! You don’t need a Ph.D. degree in these topics to get started but you do need a basic understanding.

(a) Learn Linear Algebra and Multivariate Calculus

Both Linear Algebra and Multivariate Calculus are important in Machine Learning. However, the extent to which you need them depends on your role as a data scientist. If you are more focused on application heavy machine learning, then you will not be that heavily focused on maths as there are many common libraries available. But if you want to focus on R&D in Machine Learning, then mastery of Linear Algebra and Multivariate Calculus is very important as you will have to implement many ML algorithms from scratch.

(b) Learn Statistics

Data plays a huge role in Machine Learning. In fact, around 80% of your time as an ML expert will be spent collecting and cleaning data. And statistics is a field that handles the collection, analysis, and presentation of data. So it is no surprise that you need to learn it!!! Some of the key concepts in statistics that are important are Statistical Significance, Probability Distributions,

Hypothesis Testing, Regression, etc. Also, Bayesian Thinking is also a very important part of ML which deals with various concepts like Conditional Probability, Priors, and Posteriors, Maximum Likelihood, etc.

(c) Learn Python

Some people prefer to skip Linear Algebra, Multivariate Calculus and Statistics and learn them as they go along with trial and error. But the one thing that you absolutely cannot skip is Python! While there are other languages you can use for Machine Learning like R, Scala, etc. Python is currently the most popular language for ML. In fact, there are many Python libraries that are specifically useful for Artificial Intelligence and Machine Learning such as Keras, TensorFlow, Scikit-learn, etc.

So if you want to learn ML, it's best if you learn Python! You can do that using various online resources and courses such as **Fork Python** available Free on Geeks for Geeks.

Step 2 – Learn Various ML Concepts

Now that you are done with the prerequisites, you can move on to learning ML (Which is the fun part!!!) It's best to start with the basics and then move on to the more complicated stuff. Some of the basic concepts in ML are:

(a) Terminologies of Machine Learning

- **Model** – A model is a specific representation learned from data by applying some machine learning algorithm. A model is also called a hypothesis.
- **Feature** – A feature is an individual measurable property of the data. A set of numeric features can be conveniently described by a feature vector. Feature vectors are fed as input to the model. For example, to predict some fruit, there may be features like color, smell, taste, etc.
- **Target (Label)** – A target variable or label is the value to be predicted by our model. For the fruit example discussed in the feature section, the label with each set of input would be the name of the fruit like apple, orange, banana, etc.
- **Training** – The idea is to give a set of inputs(features) and its expected outputs(labels), so after training, we will have a model (hypothesis) that will then map new data to one of the categories trained in.
- **Prediction** – Once our model is ready, it can be fed a set of inputs to which it will provide a predicted output (label).

(b) Types of Machine Learning

- **Supervised Learning** – This involves learning from a training dataset with labeled data using classification and regression models. This learning process continues until the required level of performance is achieved.
- **Unsupervised Learning** – This involves using unlabeled data and then finding the underlying structure in the data to learn more and more about the data itself using factor and cluster analysis models.
- **Semi-supervised Learning** – This involves using unlabeled data like Unsupervised Learning with a small amount of labeled data. Using labeled data vastly increases learning accuracy and is also more cost- effective than Supervised Learning.
- **Reinforcement Learning** – This involves learning optimal actions through trial and error. So, the next action is decided by learning behaviors that are based on the current state and that will maximize the reward in the future.

Advantages of Machine learning

1. Easily identifies trends and patterns

Machine Learning can review large volumes of data and discover specific trends and patterns that would not be apparent to humans. For instance, for an e-commerce website like Amazon, it serves to understand the browsing behaviors and purchase histories of its users to help cater to the right products, deals, and reminders relevant to them. It uses the results to reveal relevant advertisements to them.

2. No human intervention needed (automation)

With ML, you do not need to babysit your project every step of the way. Since it means giving machines the ability to learn, it lets them make predictions and improve the algorithms on their own. A common example of this is anti-virus software; they learn to filter new threats as they are recognized. ML is also good at recognizing spam.

3. Continuous Improvement

As ML algorithms gain experience, they keep improving in accuracy and efficiency. This lets them make better decisions. Say you need to make a weather forecast model. As the amount of data you have keeps growing, your algorithms learn to make more accurate predictions faster.

4. Handling multi-dimensional and multi-variety data

Machine Learning algorithms are good at handling data that are multi-dimensional and multi- variety, and they can do this in dynamic or uncertain environments.

5. Wide Applications

You could be an e-tailer or a healthcare provider and make ML work for you. Where it does apply, it holds the capability to help deliver a much more personal experience to customers while also targeting the right customers.

Disadvantages of Machine Learning

1. Data Acquisition

Machine Learning requires massive data sets to train on, and these should be inclusive/unbiased, and of good quality. There can also be times where they must wait for new data to be generated.

2. Time and Resources

ML needs enough time to let the algorithms learn and develop enough to fulfill their purpose with a considerable amount of accuracy and relevancy. It also needs massive resources to function. This can mean additional requirements of computer power for you.

3. Interpretation of Results

Another major challenge is the ability to accurately interpret results generated by the algorithms. You must also carefully choose the algorithms for your purpose.

4. High error-susceptibility

Machine Learning is autonomous but highly susceptible to errors. Suppose you train an algorithm with data sets small enough to not be inclusive. You end up with biased predictions coming from a biased training set. This leads to irrelevant advertisements being displayed to customers. In the case of ML, such blunders can set off a chain of errors that can go undetected for long periods of time. And when they do get noticed, it takes quite some time to recognize the source of the issue, and even longer to correct it.

Python Development Steps

Guido Van Rossum published the first version of Python code (version 0.9.0) at alt. Sources in February 1991. This release included already exception handling, functions, and the core data types of lists, dict, str and others. It was also object oriented and had a module system. Python version 1.0 was released in January 1994. The major new features included in this release were the functional programming tools lambda, map, filter and reduce, which Guido Van Rossum never liked. Six and a half years later in October 2000, Python 2.0 was introduced. This release included list comprehensions, a full garbage collector and it was supporting Unicode. Python flourished for another 8 years in versions 2.x before the next major release as Python 3.0 (also known as "Python 3000" and "Py3K") was released. Python 3 is not backwards compatible with Python 2.x. The emphasis in Python 3 had been on the removal of duplicate programming constructs and modules, thus fulfilling or coming close to fulfilling the 13th law of the Zen of Python: "There should be one and preferably only one – obvious way to do it. Some changes in Python 7.3:

- Print is now a function.
- Views and iterators instead of lists.
- The rules for ordering comparisons have been simplified. E.g. a heterogeneous list cannot be sorted, because all the elements of a list must be comparable to each other.
- There is only one integer type left, i.e. int. long is int as well.
- The division of two integers returns a float instead of an integer. "//" can be used to have the "old" behavior.
- Text Vs. Data Instead of Unicode Vs. 8-bit.

Purpose

I demonstrated that my approach enables successful segmentation of intra-retinal layer seven with low-quality images containing speckle noise, low contrast, and different intensity ranges throughout with the assistance of the ANIS feature.

Python

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace. Python features a dynamic type of system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

- Python is Interpreted – Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is like PERL and PHP.
- Python is Interactive – you can sit at a Python prompt and interact with the interpreter directly to write your programs.
- Python also acknowledges that speed of development is important. Readable and terse code is part of this, and so is access to powerful constructs that avoid tedious repetition of code. how much code you must scan, read and/or understand to troubleshoot problems or tweak behaviors.
- This speed of development, the ease with which a programmer of other languages can pick up basic Python skills and the huge standard library is key to another area where Python excels.
- All its tools have been quick to implement, saved a lot of time, and several of them have later been patched and updated by people with no Python background - without breaking.

Modules Used in Project

TensorFlow

TensorFlow is a free and open-source software library for dataflow and differentiable programming across a range of tasks. It is a symbolic math library, and is also used for machine

learning applications such as neural networks. It is used for both research and production at Google. TensorFlow was developed by the Google Brain team for internal Google use. It was released under the Apache 2.0 open-source license on November 9, 2015.

NumPy

NumPy is a general-purpose array-processing package. It provides a high-performance multidimensional array object, and tools for working with these arrays.

It is the fundamental package for scientific computing with Python. It contains various features including these important ones:

- A powerful N-dimensional array object
- Sophisticated(broadcasting) functions
- Tools for integrating C/C++ and Fortran code
- Useful linear algebra, Fourier transform, and random number capabilities

Besides its obvious scientific uses, NumPy can also be used as an efficient multi-dimensional container of generic data. Arbitrary data-types can be defined using NumPy which allows NumPy to integrate with a wide variety of databases seamlessly and speedily.

Pandas

Pandas is an open-source Python Library providing high-performance data manipulation and analysis tool using its powerful data structures. Python was majorly used for data munging and preparation. It had very little contribution towards data analysis. Pandas solved this problem. Using Pandas, we can accomplish five typical steps in the processing and analysis of data, regardless of the origin of data load, prepare, manipulate, model, and analyze. Python with Pandas is used in a wide range of fields including academic and commercial domains including finance, economics, Statistics, analytics, etc.

Matplotlib

Matplotlib is a Python 2D plotting library which produces publication quality figures in a variety of hardcopy formats and interactive environments across platforms. Matplotlib can be used in Python scripts, the Python and IPython shells, the Jupyter Notebook, web application servers, and four graphical user interface toolkits. Matplotlib tries to make easy things easy and hard things possible. You can generate plots, histograms, power spectra, bar charts, error charts, scatter plots, etc., with just a few lines of code. For examples, see the sample plots and thumbnail gallery.

For simple plotting the pyplot module provides a MATLAB-like interface, particularly when combined with IPython. For the power user, you have full control of line styles, font properties, axes properties, etc., via an object-oriented interface or via a set of functions familiar to MATLAB users.

Scikit – learn

Scikit-learn provides a range of supervised and unsupervised learning algorithms via consistent interface in Python. It is licensed under a permissive simplified BSD license and is distributed under many Linux distributions, encouraging academic and commercial use.

Python

Python is an interpreted high-level programming language for general-purpose programming. Created by Guido van Rossum and first released in 1991, Python has a design philosophy that emphasizes code readability, notably using significant whitespace.

Python features a dynamic type system and automatic memory management. It supports multiple programming paradigms, including object-oriented, imperative, functional and procedural, and has a large and comprehensive standard library.

- Python is Interpreted – Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- Python is Interactive – you can actually sit at a Python prompt and interact with the interpreter directly to write your programs.

Python also acknowledges that speed of development is important. Readable and terse code is part of this, and so is access to powerful constructs that avoid tedious repetition of code. Maintainability also ties into this may be an all but useless metric, but it does say something

about how much code you have to scan, read and/or understand to troubleshoot problems or tweak behaviors. This speed of development, the ease with which a programmer of other languages can pick up basic Python skills and the huge standard library is key to another area where Python excels. All its tools have been quick to implement, saved a lot of time, and several of them have later been patched and updated by people with no Python background - without breaking.

Install Python Step-by-Step in Windows and Mac

Python, a versatile programming language, doesn't come pre-installed on your computer devices. Python was first released in the year 1991 and until today it is a very popular high-level programming language. Its style philosophy emphasizes code readability with its not able use of great white space.

The object-oriented approach and language construct provided by Python enables programmers to write both clear and logical code for projects. This software does not come pre-packaged with Windows.

How to Install Python on Windows and Mac

There have been several updates in the Python version over the years. The question is how to install Python? It might be confusing for the beginner who is willing to start learning Python, but this tutorial will solve your query. The latest or the newest version of Python is version 3.11.2 or in other words, it is Python 3.

Before you start the installation process of Python. First, you need to know about your System Requirements. Based on your system type i.e. operating system and based processor, you must download the python version. My system type is a Windows 64-bit operating system. So, the steps below are to install python version 3.11.2 on Windows 11 device or to install Python 3. Download the Python Cheat sheet [here](#). The steps on how to install Python on Windows 10, 11 and 12 are divided into 4 parts to help understand better.

Download the Correct version into the system

Step 1: Go to the official site to download and install python using Google Chrome or another web browser OR Click on the following link: <https://www.python.org>



Fig: SCREENSHOT 1

Now, check for the latest and the correct version for your operating system.

Step 2: Click on the Download Tab.

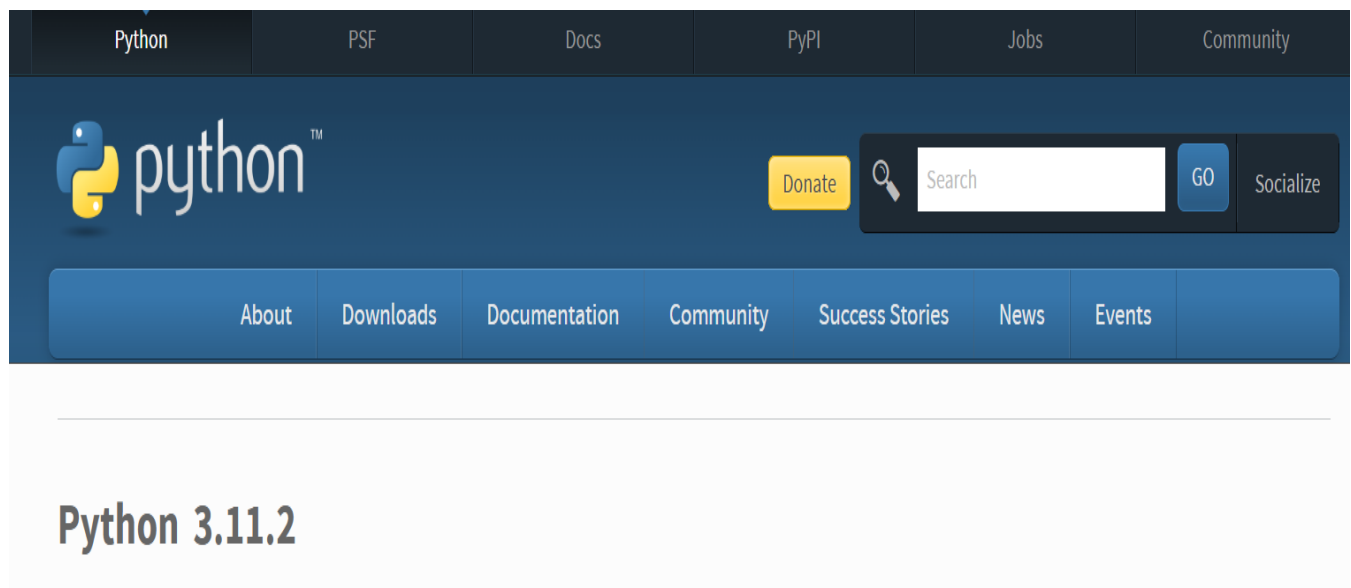


Fig: SCREENSHOT 2

Step 3: You can either select the Download Python for windows 3.11.2 button in Yellow Color or you can scroll further down and click on download with respective to their version. Here, we are downloading the most recent python version for windows 3.11.2.

Files

macOS

Download macOS 64-bit universal2 installer

Windows

Download Python install manager

Source release

Download XZ compressed source tarball

Version	Operating system	Description	File size	Sigstore	GPG	MD5 checksum
Gzipped source tarball	Source release		25.2 MB	.sigstore	SIG	f6b5226ccba5ae1ca9376aaba0b0f673
XZ compressed source tarball	Source release		19.0 MB	.sigstore	SIG	a957cffb58a89303b62124896881950b
macOS 64-bit universal2 installer	macOS	for macOS 10.9 and later	40.9 MB	CRT SIG	SIG	e038c3d5cee8c5210735a764d3f36f5a
Windows installer (64-bit)	Windows	Recommended	24.2 MB	CRT SIG	SIG	4331ca54d9eacdbe6e97d6ea63526e57
Windows installer (32-bit)	Windows		23.0 MB	CRT SIG	SIG	2123016702bbb45688baedc3695852f4
Windows installer (ARM64)	Windows	Experimental	23.3 MB	CRT SIG	SIG	040ab03501a65cc26bd340323bb1972e
Windows embeddable package (64-bit)	Windows		10.1 MB	CRT SIG	SIG	ae7de44ecbe2d3a37dbde3ce669d31b3
Windows embeddable package (32-bit)	Windows		9.1 MB	CRT SIG	SIG	64853e569d7cb0d1547793000ff9c9b6
Windows embeddable package (ARM64)	Windows		9.3 MB	CRT SIG	SIG	747090b80a52e8bcb5cb65f78fee575

Fig: SCREENSHOT 3

Step 4: Scroll down the page until you find the Files option.

Step 5: Here you see a different version of python along with the operating system.

Looking for a specific release?

Python releases by version number:

Release version	Release date	
Python 3.11.2	Feb. 8, 2023	Download
Python 3.11.1	Dec. 6, 2022	Download
Python 3.11.0	Oct. 24, 2022	Download
Python 3.10.20	March 3, 2026	Download
Python 3.10.19	Oct. 9, 2025	Download
Python 3.10.18	June 3, 2025	Download
Python 3.10.17	April 8, 2025	Download
Python 3.10.16	Dec. 3, 2024	Download

[View older releases](#)

Fig: SCREENSHOT 4

- To download Windows 32-bit python, you can select anyone from the three options: Windows x86 embeddable zip file, Windows x86 executable installer or Windows x86 web-based installer.
- To download Windows 64-bit python, you can select anyone from the three options: Windows x86-64embeddable zip file, Windows x86-64 executable installer or Windows x86-64 web-based installer.

Here we will install Windows x86-64 web-based installer. Here is your first part regarding which version of python is to be downloaded. Now we move ahead with the second part in installing python i.e. Installation.

Note: To know the changes or updates that are made in the version you can click on the Release Note Option.

Installation of Python

Step 1: Go to Download and Open the downloaded python version to carry out the installation process.

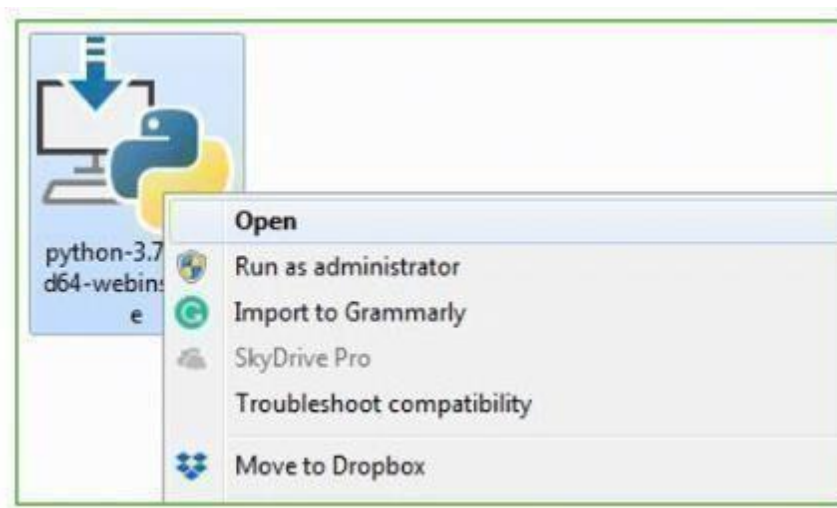


Fig: SCREENSHOT 5

Step 2: Before you click on Install Now, make sure to put a tick on Add Python 3.11.2 to PATH.



Fig: SCREENSHOT 6

Step 3: Click on Install NOW After the installation is successful. Click on Close.



Fig: SCREENSHOT 7

With these above three steps on python installation, you have successfully and correctly installed Python. Now is the time to verify the installation.

Note: The installation process might take a couple of minutes.

Verify the Python Installation

Step 1: Click on Start

Step 2: In the Windows Run Command, type “cmd”.

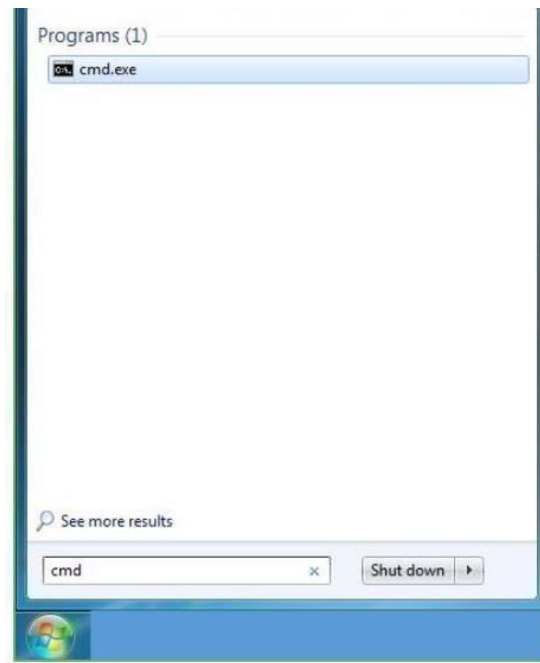


Fig: SCREENSHOT 8

Step 3: Open the Command prompt option.

Step 4: Let us test whether the python is correctly installed. Type python -V and press Enter.

```
Command Prompt
Microsoft Windows [Version 10.0.26200.8037]
(c) Microsoft Corporation. All rights reserved.

C:\Users\sivam>python --version
Python 3.11.2

C:\Users\sivam>
```

Fig: SCREENSHOT 9

Step 5: You will get the answer as 3.11.2

Note: If you have any of the earlier versions of Python already installed. You must first uninstall the earlier version and then install the new one.

Check how the Python IDLE works:

Step 1: Click on Start

Step 2: In the Windows Run command, type “python idle”.

Step 3: Click on IDLE (Python 3.7 64-bit) and launch the program

Step 4: To go ahead with working in IDLE you must first save the file. Click on File > Click on Save.

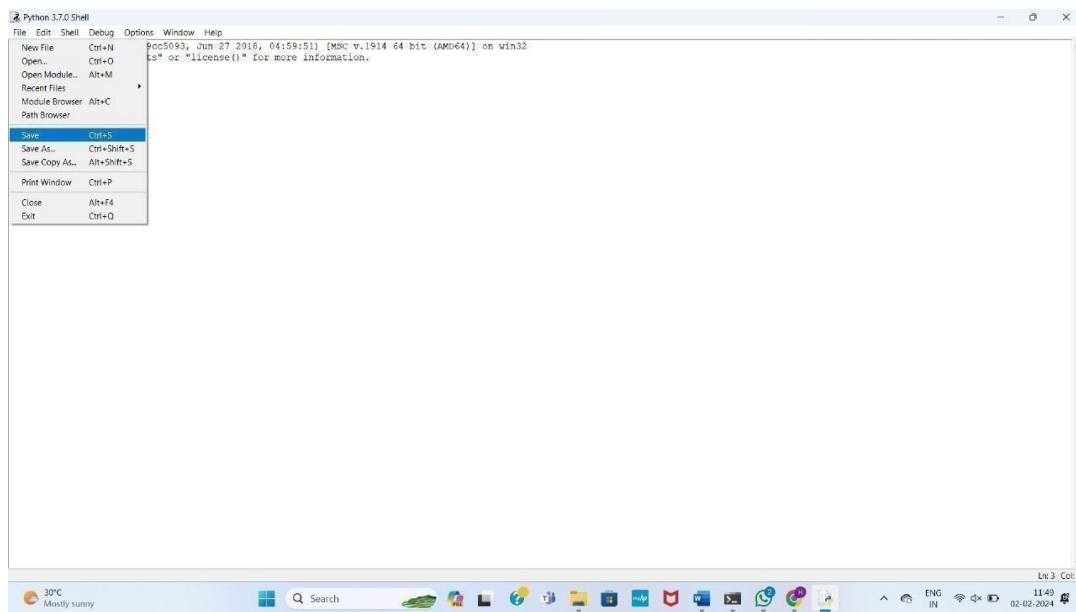


Fig: SCREENSHOT 10

Step 5: Name the file and save as type should be Python files. Click on SAVE. Here I have named the files as Hey World.

Step 6: Now for e.g. enter “print(‘Hello’)”

CHAPTER 7

SYSTEM DESIGN

7. SYSTEM DESIGN

7.1 System Architecture

The System Architecture for Twitter trend mining involves outlining the various components, their interactions, and the flow of data and processes within the system.

SYSTEM ARCHITECTURE:

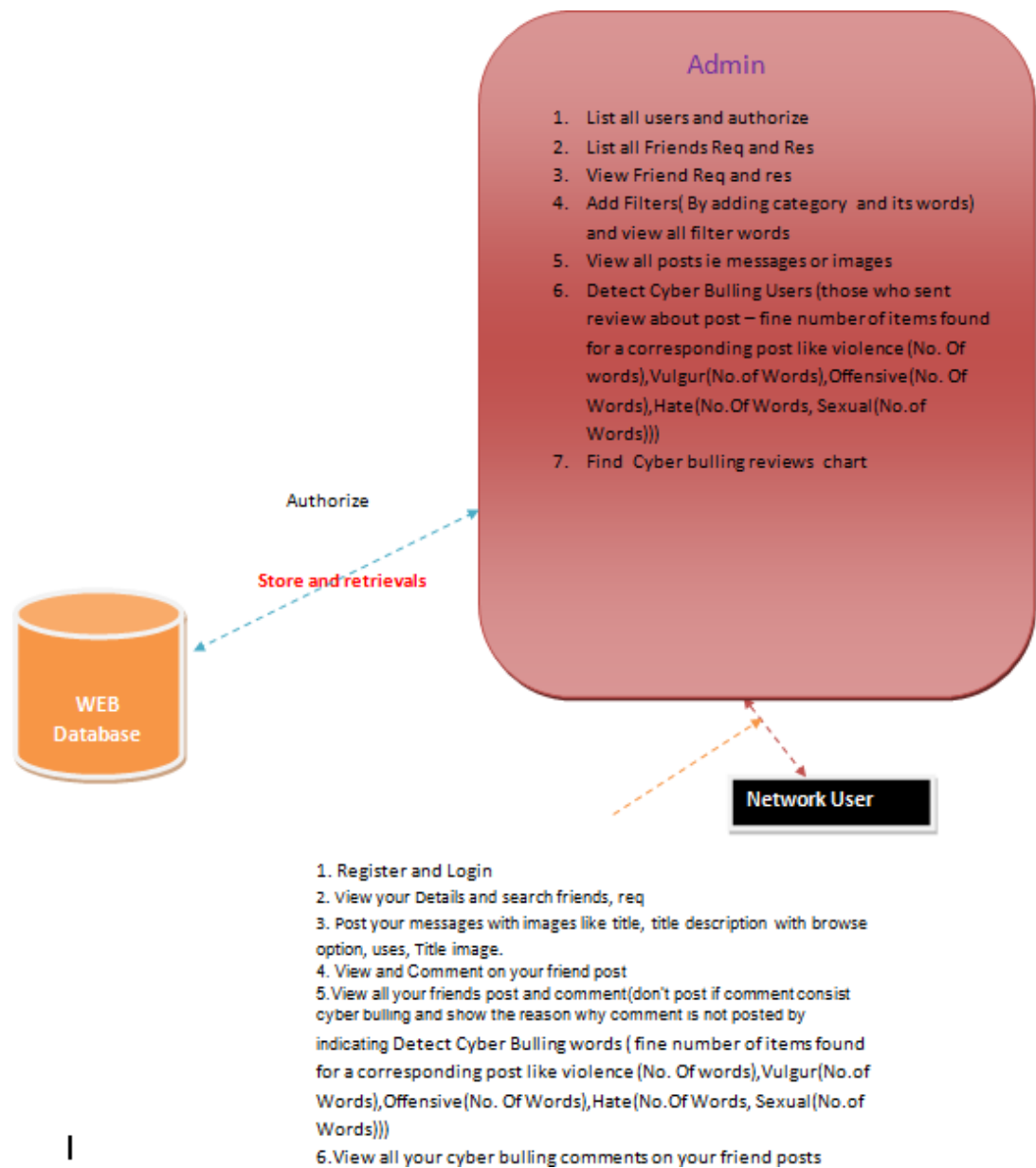
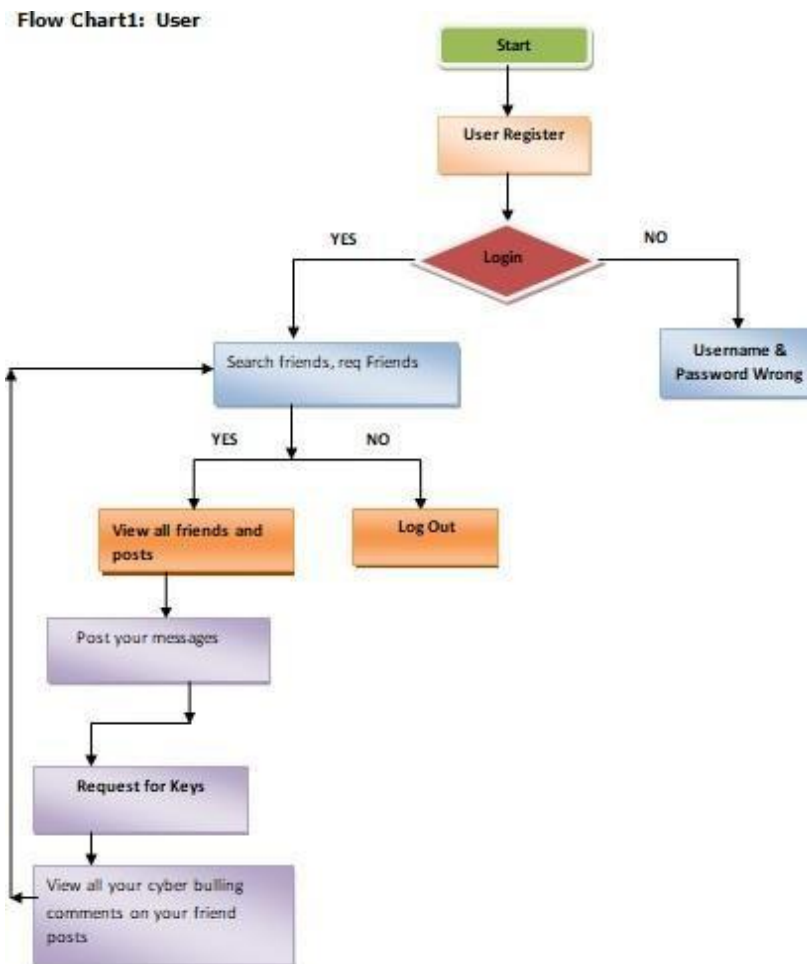


FIG-7.1.1: System Architecture

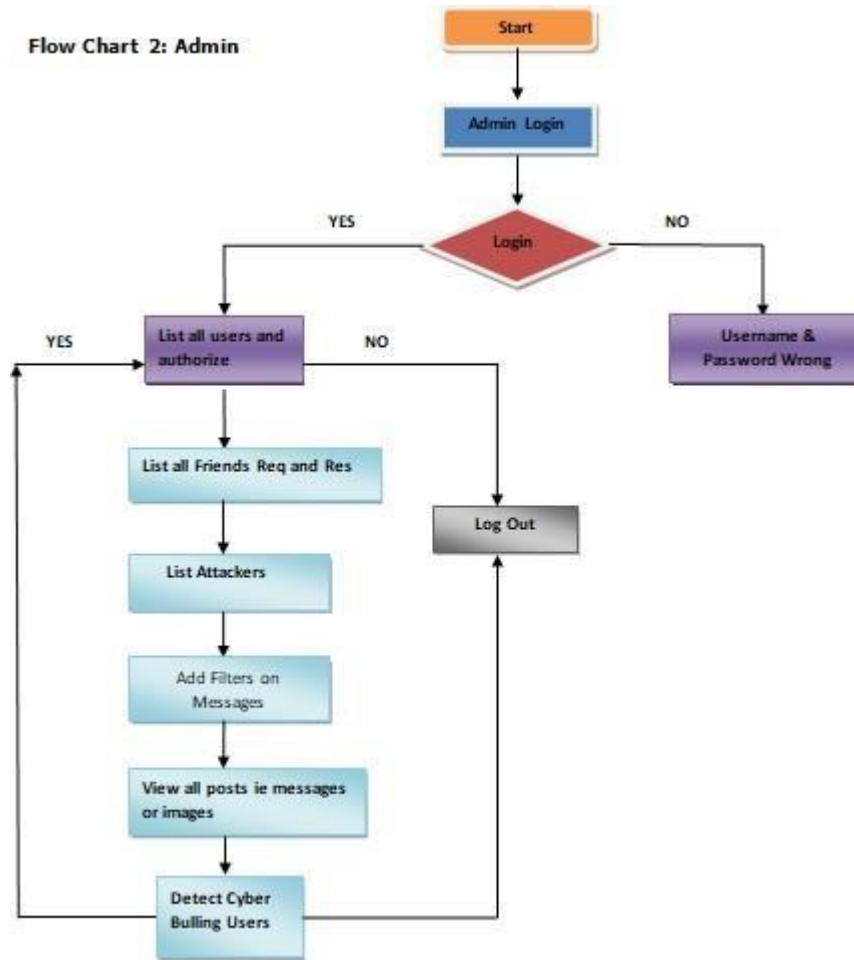
FLOWCHART

A flowchart is a type of diagram that represents a workflow or process. A flowchart can also be defined as a diagrammatic representation of an algorithm, a step-by-step approach to solving a task.



1. FIG-7.1.2: Flow Chart 1

Flow Chart 2: Admin



I

DATA FLOW DIAGRAM:

1. The DFD is also called as bubble chart. It is a simple graphical formalism that can be used to represent a system in terms of input data to the system, various processing carried out on this data, and the output data is generated by this system.
2. The data flow diagram (DFD) is one of the most important modeling tools. It is used to model the system components. These components are the system process, the data used by the process, an external entity that interacts with the system and the information flows in the system.

3. DFD shows how the information moves through the system and how it is modified by a series of transformations. It is a graphical technique that depicts information flow and the transformations that are applied as data moves from input to output.
4. DFD is also known as bubble chart. A DFD may be used to represent a system at any level of abstraction. DFD may be partitioned into levels that represent increasing information flow and functional detail.

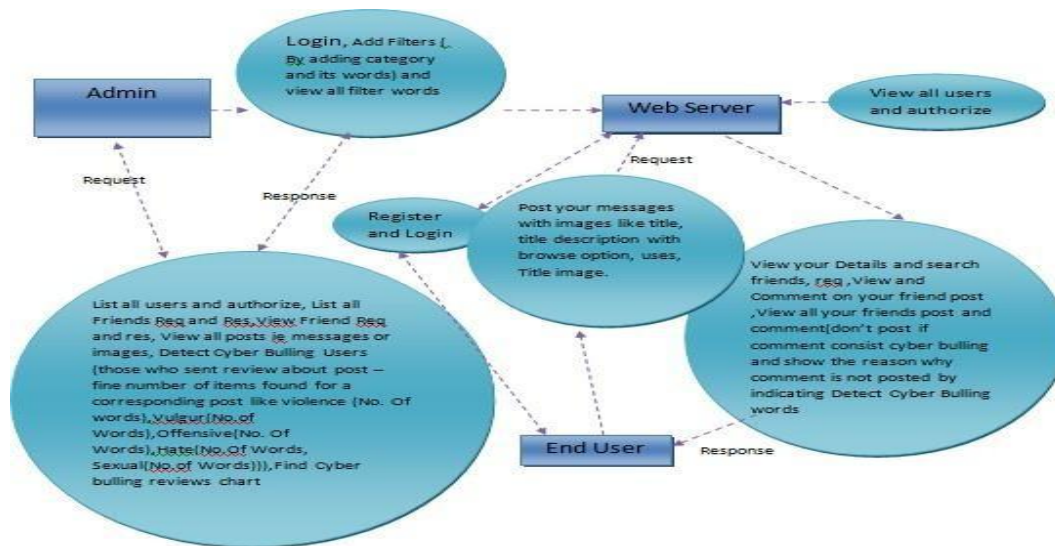


FIG-7.1.3: Data Flow Diagram

7.2 UML Diagrams

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group.

The goal is for UML to become a common language for creating models of object-oriented computer software. In its current form UML is comprised of two major components: a Meta- model and a notation. In the future, some form of method or process may also be added to; or associated with, UML.

The Unified Modeling Language is a standard language for specifying, Visualization, Constructing and documenting the artifacts of software system, as well as for business modeling and other non-software systems.

The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems.

The UML is a very important part of developing object-oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

GOALS:

The Primary goals in the design of the UML are as follows:

1. Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
2. Provide extendibility and specialization mechanisms to extend the core concepts.
3. Be independent of particular programming languages and development process.
4. Provide a formal basis for understanding the modeling language.
5. Encourage the growth of OO tools market.
6. Support higher level development concepts such as collaborations, frameworks, patterns and components.
7. Integrate best practices.

Use case Diagram

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

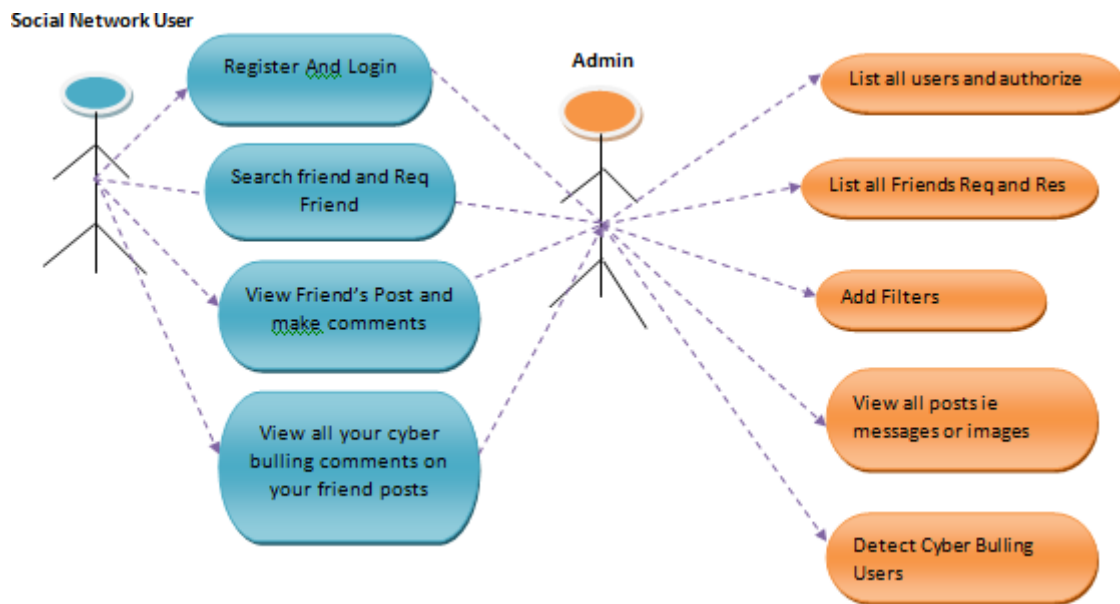


FIG-7.2.1: Use case Diagram

Class Diagram

The class diagram is used to refine the use case diagram and define a detailed design of the system. The class diagram classifies the actors defined in the use case diagram into a set of interrelated classes. The relationship or association between the classes can be either an "is-a" or "has-a" relationship. Each class in the class diagram may be capable of providing certain functionalities. These functionalities provided by the class are termed "methods" of the class. Apart from this, each class may have certain "attributes" that uniquely identify the class.

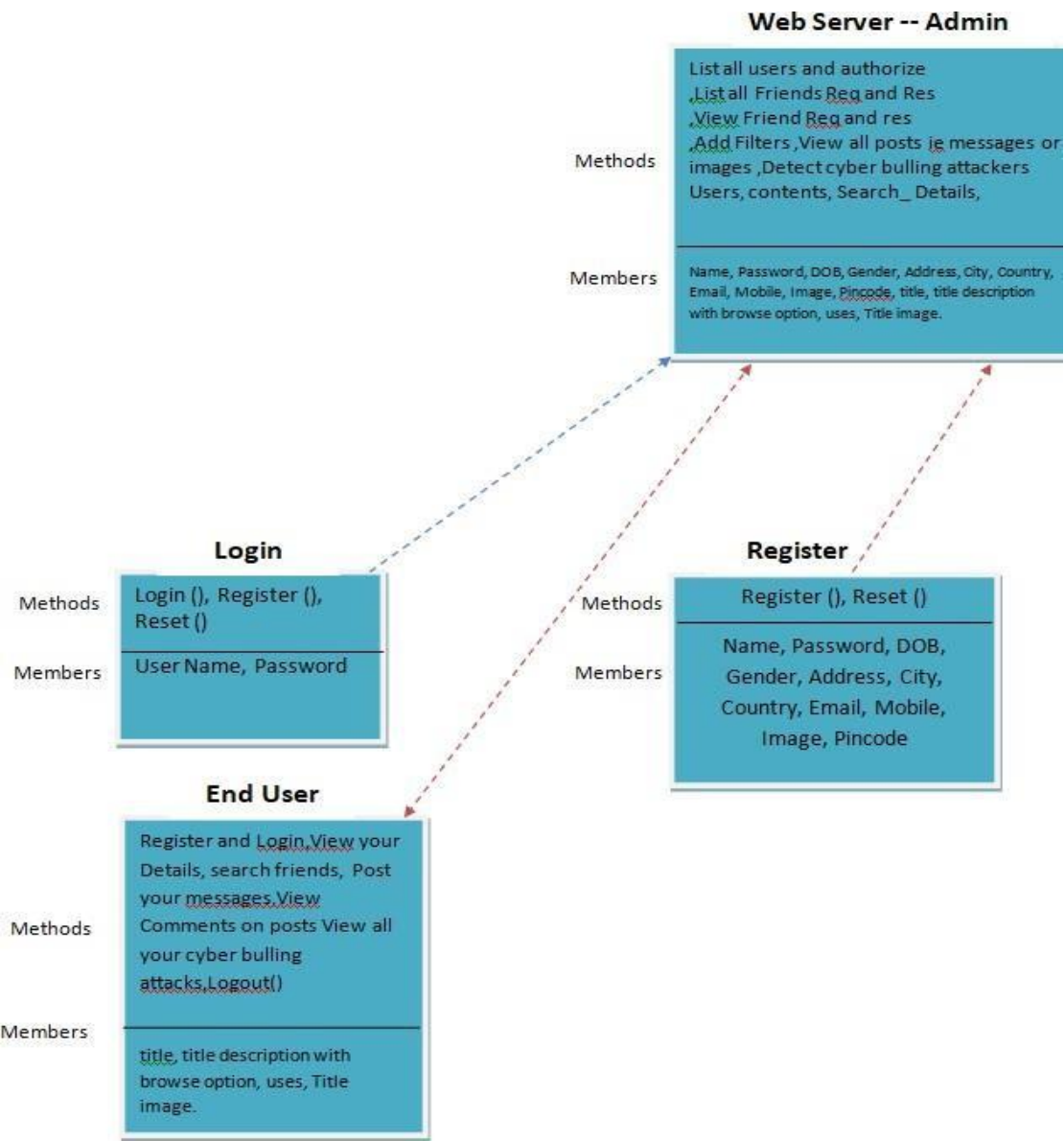


FIG-7.2.2: Class Diagram

Object Diagram

The object diagram is a special kind of class diagram. An object is an instance of a class. This essentially means that an object represents the state of a class at a given point of time while the system is running. The object diagram captures the state of different classes in the system and their relationships or associations at a given point of time.



FIG-7.2.3: Object Diagram

State Diagram

A state diagram, as the name suggests, represents the different states that objects in the system undergo during their life cycle. Objects in the system change states in response to events. In addition to this, a state diagram also captures the transition of the object's state from an initial state to a final state in response to events affecting the system.

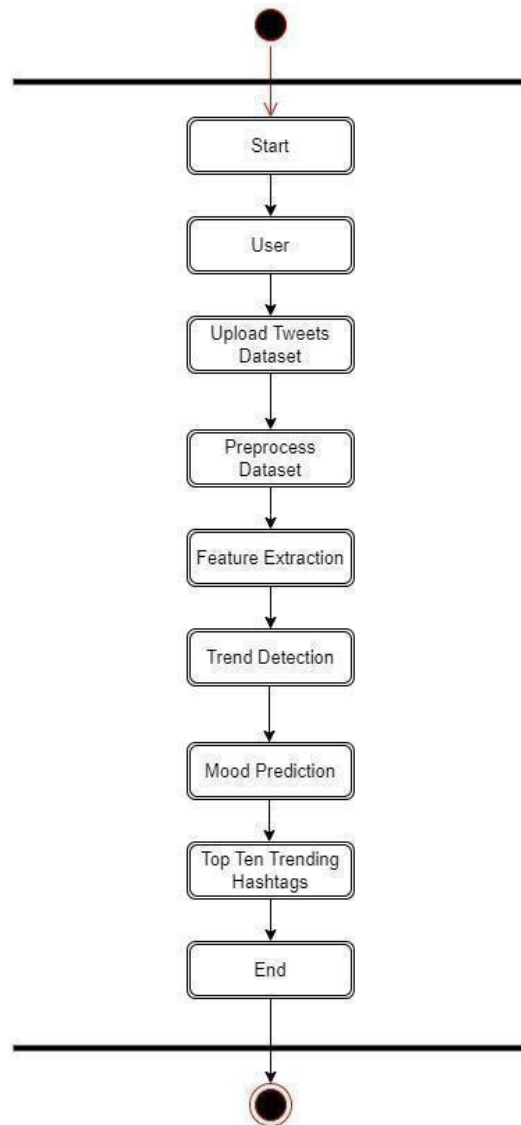


FIG-7.2.4: State Diagram

Activity Diagram

The process flows in the system are captured in the activity diagram. Like a state diagram, an activity diagram also consists of activities, actions, transitions, initial and final states, and guard conditions.

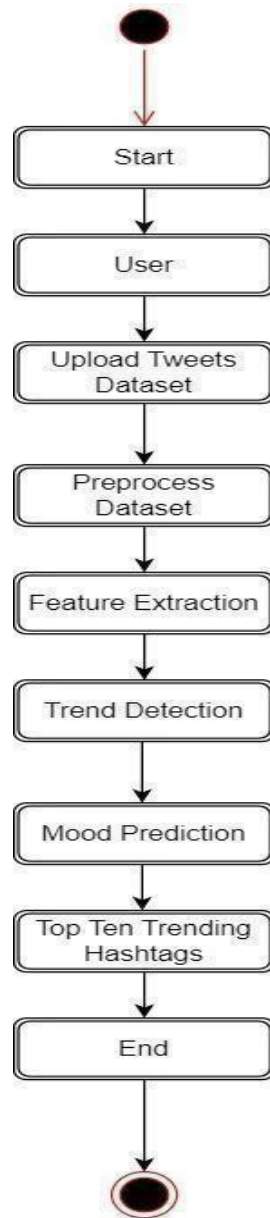


FIG-7.2.5: Activity Diagram

Sequence Diagram

A sequence diagram represents the interaction between different objects in the system. The important aspect of a sequence diagram is that it is time-ordered. This means that the exact sequence of the interactions between the objects is represented step by step. Different objects in the sequence diagram interact with each other by passing "messages".

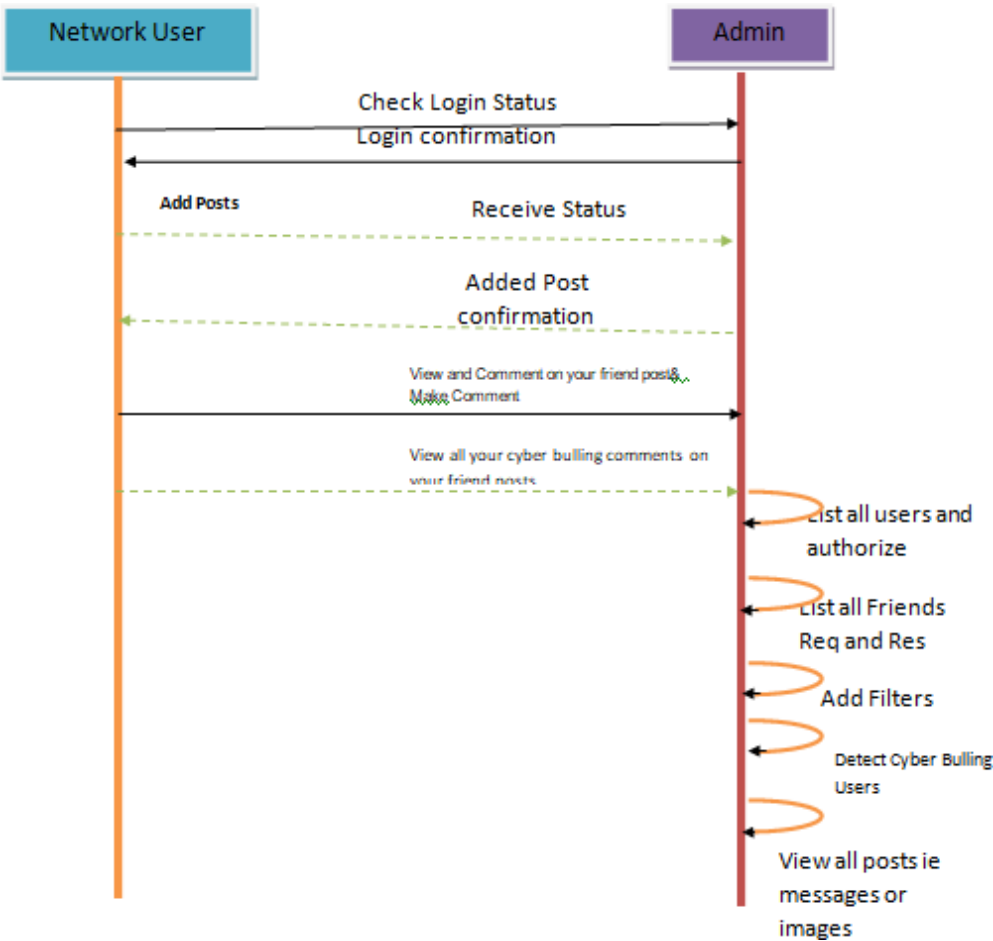


FIG-7.2.6: Sequence Diagram

Collaboration Diagram

A collaboration diagram groups together the interactions between different objects. The interactions are listed as numbered interactions that help to trace the sequence of the interactions. The collaboration diagram helps to identify all the possible interactions that each object has with other objects.

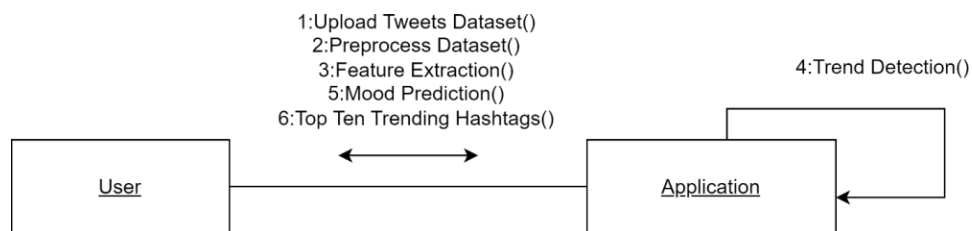


FIG-7.2.7: Collaboration Diagram

CHAPTER 8

IMPLEMENTATION

8. IMPLEMENTATION

8.1 Modules

• Admin

In this module, the Admin has to login by using valid user name and password. After login successful he can perform some operations such as view and authorize users, view all friends request and responses, Add and View Filters, View all posts, Detect Cyber Bullying Users, Find Cyber Bullying Reviews Chart.

Viewing and Authorizing Users

In this module, the admin views all users details and authorize them for login permission. User Details such as User Name, Address, Email Id, Mobile Number.

Viewing all Friends Request and Response

In this module, the admin can see all the friends' requests and response history. Details such as Requested User Name and Image, and Requested to User Name and Image, status and date.

Add and View Filters

In this module, the admin can add filters (like Violence, Vulgar, Offensive, Hate, and Sexual) as Categories with the words those related to corresponding filters.

View all posts

In this module, the admin can see all the posts added by the users with post details like post name, description and post image.

Detect Cyber Bullying Users

In this module, the admin can see all the Cyber Bullying Users (The users who had posted a comment on posts using cyber bullying words which are all listed by the admin to detect and filter). In this, the results shown as, Number of items found for a corresponding post like Violence (no. of words belongs to Violence Filter used in comments by the users), Vulgar (no. of words belongs to Vulgar Filter used in comments by the users), Offensive (no. of words belongs to Offensive Filter used in comments by the users), Hate (no. of words belongs to Hate Filter used in comments by the users), Sexual (no. of words belongs to Sexual Filter used in comments by the users).

Find Cyber Bullying Reviews Chart

In this module, the admin can see all the posts with number of cyber bullying comments posted by users for particular post.

• User

In this module, there are n numbers of users are present. User should register before performing any operations. Once user registers, their details will be stored to the database. After registration successful, he has to login by using authorized user name and password. Once Login is successful user can perform some operations like viewing their profile details, searching for friends and sending friend requests, Posting Your Messages as Posts by giving details, View and Comment on Friend Posts, viewing all friends posts and comment, view all your cyber bullying comments on your friend posts.

Viewing Profile Details, Search and Request Friends

In this module, the user can see their own profile details, such as their address, email, mobile number, profile Image.

The user can search for friends and can send friend requests or can accept friend requests. Add Posts

In this, the user can add their own posts by giving post details such as, post title, description, uses, and image of post.

View and Comment on Your Friends Post

In this, the user can see his entire friend's post details (post title, description, uses, creator and image of post) and can comment on posts.

View all Friends Posts and Comment (Cyber bullying Related)

In this, the user can see his all-friend's post details (post title, description, uses, creator and image of post) and can comment on posts.

Don't Post If the comment consists of Cyber bullying words and Shows the reason why comment is not posted by indicating Detected Cyber Bullying Words like Numbers of Cyber Bullying words Related to Filter Violence found in comment, Numbers of Cyber Bullying words Related to Filter Vulgar found in comment, Numbers of Cyber Bullying words Related to Offensive found in comment, Numbers of Cyber Bullying words Related to Hate found in comment, Numbers of Cyber Bullying words Related to Sexual found in comment,

View all Your Cyber bullying comments on your friend posts

The user can see all his posted cyber bullying comments on their friend created posts.

8.2 Source Code

```
from django.shortcuts import render

from django.template import
RequestContext from django.contrib
import messages

import pymysql

from django.http import HttpResponse
from django.conf import settings

from django.core.files.storage import FileSystemStorage import datetime
import matplotlib.pyplot as plt import re

import numpy as np

from sklearn import svm

import pandas as pd

from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
import nltk

from nltk.corpus import stopwords

from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
from sklearn.ensemble import RandomForestClassifier

from sklearn.naive_bayes import MultinomialNB

from sklearn.tree import DecisionTreeClassifier

from sklearn.neighbors import KNeighborsClassifier from sklearn.ensemble import
BaggingClassifier

#from sklearn_extensions.extreme_learning_machines.elm import GenELMClassifier

#from sklearn_extensions.extreme_learning_machines.random_layer import
RBFRandomLayer, MLPRandomLayer

from sklearn.linear_model import LogisticRegression

from django.core.files.storage import FileSystemStorage

import datetime

from numpy.linalg import norm f

rom numpy import dot

import os
```

```
import pickle

global classifier global label_count global X

global Y corpus = []

def index(request):

    if request.method == 'GET':
        return render(request, 'index.html', {})
    def SendPost(request):

        if request.method == 'GET':

            return render(request, 'SendPost.html', {}) else:
        return render(request, 'SendPost.html', {})
    def Register(request):

        if request.method == 'GET':
            return render(request, 'Register.html', {})
        def Admin(request):

            if request.method == 'GET':
                return render(request, 'Admin.html', {})
            def Login(request):

                if request.method == 'GET':
                    return render(request, 'Login.html', {})
                def VerifyUsers(request):

                    if request.method == 'GET':
                        return render(request, 'VerifyUsers.html', {})
            def AddCyberMessages(request): if request.method == 'GET':

                return render(request, 'AddCyberMessages.html', {})
        def RunAlgorithms(request): if request.method == 'GET':

            return render(request, 'RunAlgorithms.html', {})
        def MonitorPost(request):

            if request.method == 'GET':
                posts = []

                strdata = '<table border=1 align=center width=100%><tr><th>Sender Name</th><th>File
Name</th><th>Message</th><th>Post Time</th> <th>Status</th></tr><tr>'
```

```
con = pymysql.connect(host='127.0.0.1',port = 3306,user = 'root', password = "", database =
'cyber',charset='utf8') with con:
    cur = con.cursor() cur.execute("select * FROM posts") rows = cur.fetchall()

for row in rows: post = {
    'sender_name': row[0], 'file_name': "/photo/"+str(row[1]), 'message': row[2],

    'post_time': row[3],
    'status': row[4]
}

    posts.append(post) context = {'posts': posts}
    return render(request, 'MonitorPost.html', context) def AddBullyingWords(request):

if request.method == 'POST':

message = request.POST.get('t1', False) label = request.POST.get('t2', False) message =
message.strip('\n')

message = message.strip() message = message.lower()
message = re.sub(r'^a-zA-Z\s+', "", message) file = open('dataset.txt','a+')
file.write(message+"", "+label+"\n") file.close()

context= {'data': 'Cyber Words added to dataset as '+label} return render(request,
'AddCyberMessages.html', context)

def Signup(request):
if request.method == 'POST':
username = request.POST.get('t1', False) password = request.POST.get('t2', False) contact =
request.POST.get('t3', False) email = request.POST.get('t4', False) address =
request.POST.get('t5', False)

    db_connection = pymysql.connect(host='127.0.0.1',port = 3306,user = 'root', password = "",
database = 'cyber',charset='utf8')
    db_cursor = db_connection.cursor()
        student_sql_query = "INSERT INTO
users(username,password,contact_no,email,address,status)
VALUES('"+username+"','"+password+"','"+contact+"','"+email+"','"+address+"','pending')"

    db_cursor.execute(student_sql_query)

    db_connection.commit()

    print(db_cursor.rowcount, "Record Inserted")
```

```
if db_cursor.rowcount == 1:

    context= {'data':'Signup Process Completed'} return render(request, 'Register.html', context)
else:

    context= {'data':'Error in signup process'} return render(request, 'Register.html', context)
def UserLogin(request):

    if request.method == 'POST':
        username = request.POST.get('t1', False) password = request.POST.get('t2', False) index = 0

        con = pymysql.connect(host='127.0.0.1',port = 3306,user = 'root', password = "", database =
        'cyber',charset='utf8') with con:

            cur = con.cursor() cur.execute("select * FROM users") rows = cur.fetchall()

            for row in rows:
                if row[0] == username and password == row[1] and row[5] == 'Accepted': index = 1

                    break if index == 1:
                        file = open('session.txt','w') file.write(username) file.close()

                        context= {'data':'welcome '+username}
                        return render(request, 'UserScreen.html', context) else:
                            context= {'data':'login failed'}
                            return render(request, 'Login.html', context)

def AdminLogin(request):
    if request.method == 'POST':
        username = request.POST.get('t1', False) password = request.POST.get('t2', False)

        if username == 'admin' and password == 'admin':
            context= {'data':'welcome '+username}
            return render(request, 'AdminScreen.html', context) else:
                context= {'data':'login failed'}
                return render(request, 'Admin.html', context)
def ViewUsers(request):

    if request.method == 'GET':
        strdata = '<table border=1 align=center
        width=100%><tr><th>Username</th><th>Password</th><th>Contact No</th><th>Email
        ID</th><th>Address</th><th>Status</th></tr><tr>'

        con = pymysql.connect(host='127.0.0.1',port = 3306,user = 'root', password = "", database =
        'cyber',charset='utf8') with con:
```

```
cur = con.cursor() cur.execute("select * FROM users") rows = cur.fetchall()

for row in rows:

    strdata+='<td>'+row[0]+'</td><td>'+row[1]+'</td><td>'+row[2]+'</td><td>'+str(row[3])+'</td>
    ><td>'+str(row[4])+'<
    /td><td>'+row[5]+'</td></tr>' context= {'data':strdata}
    return render(request, 'ViewUsers.html', context)
def UserVerification(request): if request.method == 'POST':
    username = request.POST.get('t1', False) status = request.POST.get('t2', False)
    db_connection = pymysql.connect(host='127.0.0.1',port = 3306,user = 'root', password = ",
    database = 'cyber',charset='utf8')

    db_cursor = db_connection.cursor()
    student_sql_query = "update users set status = '"+status+"'' where username = '"+username+"'"
    db_cursor.execute(student_sql_query)
    db_connection.commit() print(db_cursor.rowcount, "Record updated")
    context= {'data': 'Account status changed to : '+status} return render(request, 'VerifyUsers.html',
    context)
def ViewUserPost(request):

if request.method == 'GET': posts = []
    con = pymysql.connect(host='127.0.0.1', port=3306, user='root', password="", database='cyber',
    charset='utf8') # Get the current working directory
    current_directory = os.getcwd() with con:

    cur = con.cursor() cur.execute("select * FROM posts") rows = cur.fetchall()

for row in rows: post = {

    'sender_name': row[0], 'file_name': "/photo/"+str(row[1]), 'message': row[2],

    'post_time': row[3],
    'status': row[4]

    }

    posts.append(post) context = {'posts': posts}
    print("Current working directory:", current_directory) return render(request,
    'ViewUserPost.html', context)
```

```
def word_count(str): counts = dict() words = str.split()

for word in words: if word in counts:

    counts[word] += 1 else:
counts[word] = 1

return counts
def prediction(X_test, cls): #prediction done here y_pred = cls.predict(X_test)
    for i in range(len(X_test)):
        print("X=%s, Predicted=%s" % (X_test[i], y_pred[i])) return y_pred
def cal_accuracy(y_test, y_pred, details): msg = "

cm = confusion_matrix(y_test, y_pred) accuracy = accuracy_score(y_test,y_pred)*100
msg+=details+"<br/>"

msg+="Accuracy : "+str(accuracy)+"<br/>"
#msg+="Report : "+str(classification_report(y_test, y_pred))+"<br/>" #msg+="Confusion
Matrix : "+str(cm)+"<br/>"

return msg
def classifyPost(vec1, vec2): vector1 = np.asarray(vec1) vector2 = np.asarray(vec2)

return dot(vector1, vector2)/(norm(vector1)*norm(vector2))
def PostSent(request):

if request.method == 'POST' and request.FILES['t2']: file_path = 'model_and_corpus.pkl'
with open(file_path, 'rb') as file: data = pickle.load(file)

# Access the model and corpus classifier = data["model"] corpus = data["corpus"]

output = "
myfile = request.FILES['t2']
msg = request.POST.get('t1', False) fs = FileSystemStorage() current_directory = os.getcwd()

filename = fs.save(current_directory+'/static/photo/'+str(myfile).replace(" ", "_"), myfile) now =
datetime.datetime.now()

current_time = now.strftime("%Y-%m-%d %H:%M:%S") text = 'msg,label\n'+str(msg)+",?"
print("text is :",text)

with open("test.txt", "w") as f: f.write(text) # Write the text to the file

df = pd.read_csv('test.txt') print(df, "<- df is")
X1 = df.iloc[:, :-1].values
print(X1, "<- X1 is") dataset = "
```

```
for k in range(len(corpus)): dataset+=corpus[k]+","

dataset = dataset[0:len(dataset)-1] dataset+="\n"

print(dataset, "<- dataset is")

for i in range(len(X1)):
line = str(X1[i]).strip("\n") line = line.strip()
line = line.lower()
line = re.sub(r'^a-zA-Z\s|'+', ', line) print(line)
wordCount = word_count(line.strip()) value = "
for j in range(len(corpus)):
if corpus[j] in wordCount.keys(): value+=str(wordCount[corpus[j]])+", "
else:
value+="0,"

value = value[0:len(value)-1] dataset+=value+"\n"
print(dataset, "<- dataset2 is") f = open("test.txt", "w") f.write(dataset)

f.close()
#test = pd.read_csv("test.txt")
test = pd.read_csv("test.txt", encoding="utf-8") X1 = test.values[:, 0:label_count]
result = classifier.predict(X1) #classify = 0

#score = 0
#for i in range(len(X)):
#    cosine = classifyPost(X[i], X1) # if cosine > score:
#    score = cosine
#    classify = Y[i] status = 'Non-Bullying' print(result)

if result[0] == 0:
    status = 'Non-Bullying' else:
status = 'Bullying'

user = "

with open("session.txt", "r") as file:
for line in file:
user = line.strip("\n")
```

```
db_connection = pymysql.connect(host='127.0.0.1',port = 3306,user = 'root', password = "",
database = 'cyber',charset='utf8')
db_cursor = db_connection.cursor()

student_sql_query = "INSERT INTO posts(sender,filename,msg,posttime,status)
VALUES('"+user+"','"+str(myfile)+"','"+msg+"','"+current_time+"','"+status+"')"
db_cursor.execute(student_sql_query) db_connection.commit() print(db_cursor.rowcount,
"Record Inserted") if db_cursor.rowcount == 1:

context= {'data':'Posts details added'}

return render(request, 'SendPost.html', context) else:

context= {'data':'Error in adding post details'} return render(request, 'SendPost.html', context)
def RunAlgorithm(request): global classifier
global label_count global X
global Y
msg = ""

if request.method == 'POST':
name = request.POST.get('t1', False) stop_words = set(stopwords.words('english'))
corpus.clear()

posts = []
df= pd.read_csv('dataset.txt') X = df.iloc[:, :-1].values
Y = df.iloc[:, -1].values for i in range(len(Y)):
if Y[i] == 'Non-Bullying':
Y[i] = 0
else:
Y[i]= 1

for i in range(len(X)):
line = str(X[i]).strip('\n') line = line.strip()
line = line.lower()
line = re.sub(r'^[a-zA-Z\s]+' , "", line) arr = line.split(" ")
for k in range(len(arr)):
word = arr[k].strip("\n").strip()
if len(word) > 2 and word not in corpus and word not in stop_words: corpus.append(word)
```



```
posts.append(line) dataset = "
for k in range(len(corpus)): dataset+=corpus[k]+", "
dataset+='Label\n'

for k in range(len(posts)): text = posts[k];
wordCount = word_count(text.strip()) for j in range(len(corpus)):
if corpus[j] in wordCount.keys(): dataset+=str(wordCount[corpus[j]])+", "

else:
dataset+="0," dataset+=str(Y[k])+"\n"

f= open("features.txt", "w") f.write(dataset)

f.close()

train = pd.read_csv("features.txt") cols = train.shape[1]
features = cols - 2 label = cols - 1 label_count = label

X = train.values[:, 0:label]

Y = train.values[:, label] print(Y)
X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size = 0.2, random_state = 0)
output = ""
if name == "SVM Algorithm":
cls = svm.SVC(C=2.0,gamma='scale',kernel = 'rbf', random_state = 2) cls.fit(X_train, y_train)
prediction_data = prediction(X_test, cls)
msg = cal_accuracy(y_test, prediction_data,'SVM Accuracy') output = 'SVM Algorithm Output
details<br/><br/>'+msg classifier = cls

if name == "Decision Tree":

cls = DecisionTreeClassifier(max_depth=None, min_samples_split=2,random_state=0)
cls.fit(X_train, y_train)
prediction_data = prediction(X_test, cls)
msg = cal_accuracy(y_test, prediction_data,'Decision Tree Accuracy') output = 'Decision Tree
Algorithm Output details<br/><br/>'+msg
```

```
if name == "KNearest Neighbors":
    cls = BaggingClassifier(KNeighborsClassifier(),max_samples=0.5, max_features=0.5)
    cls.fit(X_train, y_train)
    prediction_data = prediction(X_test, cls)
    msg = cal_accuracy(y_test, prediction_data,'KNearest Neighbor Accuracy') output = 'KNearest
    Neighbor Algorithm Output details<br/><br/>'+msg

if name == "Random Forest":
    cls = RandomForestClassifier(n_estimators=1,max_depth=1,random_state=None)
    cls.fit(X_train, y_train)

    prediction_data = prediction(X_test, cls)
    msg = cal_accuracy(y_test, prediction_data,'Random Forest Accuracy') output = 'Random
    Forest Algorithm Output details<br/><br/>'+msg

if name == "Naive Bayes": cls = MultinomialNB() cls.fit(X_train, y_train)

    prediction_data = prediction(X_test, cls)
    msg = cal_accuracy(y_test, prediction_data,'Naive Bayes Accuracy') output = 'Naive Bayes
    Algorithm Output details<br/><br/>'+msg

# if name == "Extreme Machine Learning":

#     log_reg = LogisticRegression(class_weight='balanced')
#     srhl_tanh = MLPRandomLayer(n_hidden=1500, activation_func='tanh') # cls =
#     GenELMClassifier(hidden_layer=srhl_tanh,regressor=log_reg)
#     cls.fit(X_train, y_train)
#     prediction_data = prediction(X_test, cls)
#     #msg = cal_accuracy(y_test, prediction_data,'Extreme Machine Learning Algorithm
#     Accuracy, Classification Report & Confusion Matrix')
#     cm = confusion_matrix(y_test, prediction_data)
#     accuracy = 98.5 #accuracy_score(y_test,y_pred)*100

#     msg+="Extreme Machine Learning Algorithm Accuracy<br/>" # msg+="Accuracy :
#     "+str(accuracy)+"<br/>"

#     #msg+="Report : "+str(classification_report(y_test, prediction_data))+"<br/>" #
#     #msg+="Confusion Matrix : "+str(cm)+"<br/>"

#output = 'Extreme Machine Learning Algorithm Output details<br/><br/>'+msg classifier = cls
```

```
context= {'data':" "+output}
# File path to save both objects file_path = 'model_and_corpus.pkl' # Save the model and
corpus to a file with open(file_path, 'wb') as file:

    pickle.dump({"model": classifier, "corpus": corpus}, file) return render (request,
'RunAlgorithms.html', context)
```

CHAPTER 9

SYSTEM TESTING

9.SYSTEM TESTING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub-assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of tests. Each test type addresses a specific testing requirement.

What do you mean by software testing?

Testing involves operation of a system or application under controlled conditions and evaluating the results. The controlled conditions should include both normal and abnormal conditions. Testing should intentionally attempt to make things go wrong to determine if things happen when they shouldn't or things don't happen when they should. It is oriented to 'detection'.

Unit Testing:

Unit testing is a software development process in which the smallest testable parts of an application, called units, are individually and independently scrutinized for proper operation. Unit testing is often automated but it can also be done manually. This testing mode is a component of Extreme Programming (XP), a pragmatic method of software development that takes a meticulous approach to building a product by means of continual testing and revision.

Unit tests are written from a programmer's perspective. They ensure that a particular method of a class successfully performs a set of specific tasks. Each test confirms that a method produces the expected output when given a known input.

Performance Testing:

Performance testing is the process of determining the speed or effectiveness of a computer, network, software program or device. This process can involve quantitative tests done in a lab, such as measuring the response time or the number of MIPS (millions of instructions per second) at which a system functions. Qualitative attributes such as

Reliability, scalability and interoperability may also be evaluated. Performance testing is often done in conjunction with stress testing.

Performance testing can verify that a system meets the specifications claimed by its manufacturer or vendor. The process can compare two or more devices or programs in terms of parameters such as speed, data transfer rate, bandwidth, throughput, efficiency or reliability.

Performance testing can also be used as a diagnostic aid in locating communications bottlenecks. Often a system will work much better if a problem is resolved at a single point or in a single component. For example, even the fastest computer will function poorly on today's Web if the connection occurs at only 40 to 50 Kbps (kilobits per second).

Integration Testing:

Integration testing, also known as integration and testing (I&T), is a software development process which program units are combined and tested as groups in multiple ways. In this context, a unit is defined as the smallest testable part of an application. Integration testing can expose problems with the interfaces among program components before trouble occurs in real-world program execution. Integration testing is a component of Extreme Programming (XP), a pragmatic method of software development that takes a meticulous approach to building a product by means of continual testing and revision.

Test case1:

Test case for Login form:

FUNCTION:	LOGIN
EXPECTED RESULTS:	Should Validate the user and check his existence in database
ACTUAL RESULTS:	Validate the user and checking the user against the database
LOW PRIORITY	No
HIGH PRIORITY	Yes

Test case2:**Test case for User Registration form:**

FUNCTION:	USER REGISTRATION
EXPECTED RESULTS:	Should check if all the fields are filled by the user and saving the user to database.
ACTUAL RESULTS:	Checking whether all the fields are field by user or not through validations and saving user.
LOW PRIORITY	No
HIGH PRIORITY	Yes

Test case3:**Test case for Change Password:**

When the old password does not match with the new password, then this results in displaying an error message as “OLD PASSWORD DOES NOT MATCH WITH THE NEW PASSWORD”.

FUNCTION:	Change Password
EXPECTED RESULTS:	Should check if old password and new password fields are filled by the user and saving the user to database.
ACTUAL RESULTS:	Checking whether all the fields are field by user or not through validations and saving user.
LOW PRIORITY	No
HIGH PRIORITY	Yes

Test case 4:**Test case for Forget Password:**

When a user forgets his password, he is asked to enter Login name, ZIP code, Mobile number. If these are matched with the already stored ones then user will get his original password.

Module	Functionality	Test Case	Expected Results	Actual Results	Result	Priority
User	Login Use case	1. Navigate To www.Sample.Com 2. Click On Submit Button Without Entering Username and Password	A Validation Should Be as Below “Please Enter Valid Username & Password”	A Validation Has Been Populated as Expected	Pass	High
		1. A Navigate To www.Sample.Com 1. 2. Click On Submit Button with Out Filling Password	A Validation Should Be as Below “Please Enter Valid Password or Password Field Can Not Be Empty “	A Validation Is Shown as Expected	Pass	High

		and with Valid Username				
		Test Username Field				
		1. Navigate To www.Sample.Com 2. Enter Both Username and Password Wrong and Hit Enter	A Validation Shown as Below “The Username Entered Is Wrong”	A Validation Is Shown as Expected	Pass	High
		1. Navigate To www.Sample.Com 2. Enter Validate Username and Password and Click on Submit	Validate Username and Password in Data Base and Once If They Correct Then Show the Main Page	Main Page/ Home Page Has Been Displayed	Pass	High

CHAPTER 10

SCREENSHOTS

10.SCREENSHOTS

To run project, in visual studio run the command “python manage.py runserver” then, it will run at <http://127.0.0.1:8000/> server.

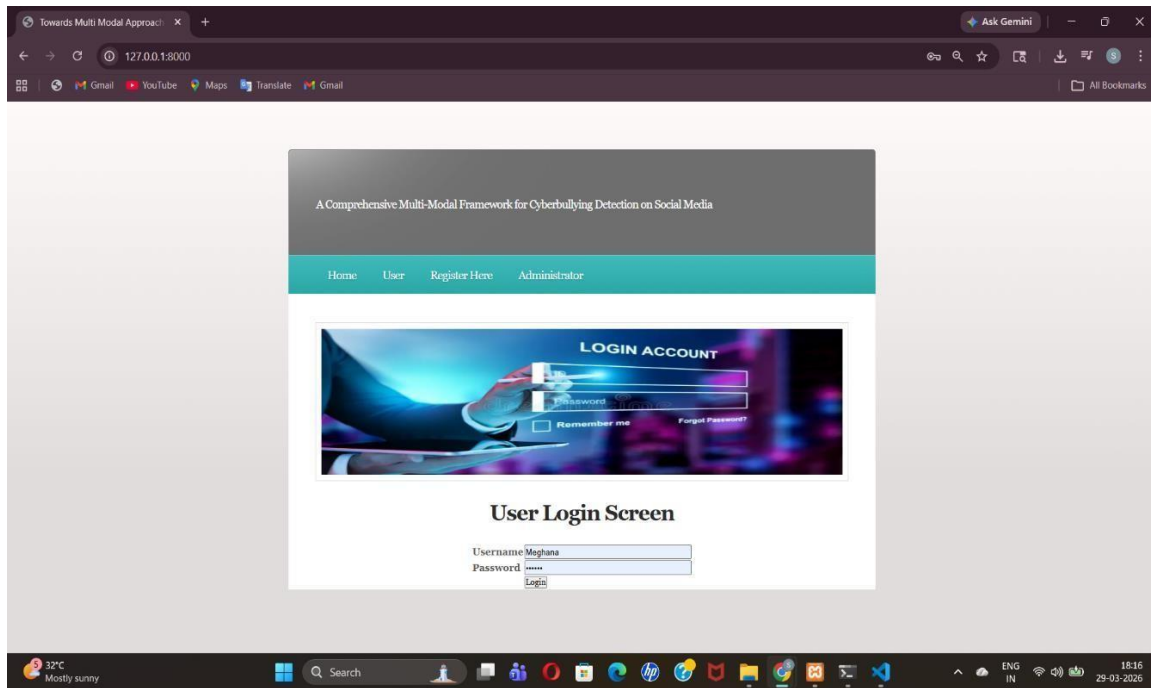


Fig: SCREENSHOT 11

Here we can see user login screen. If you are old user login to the application.

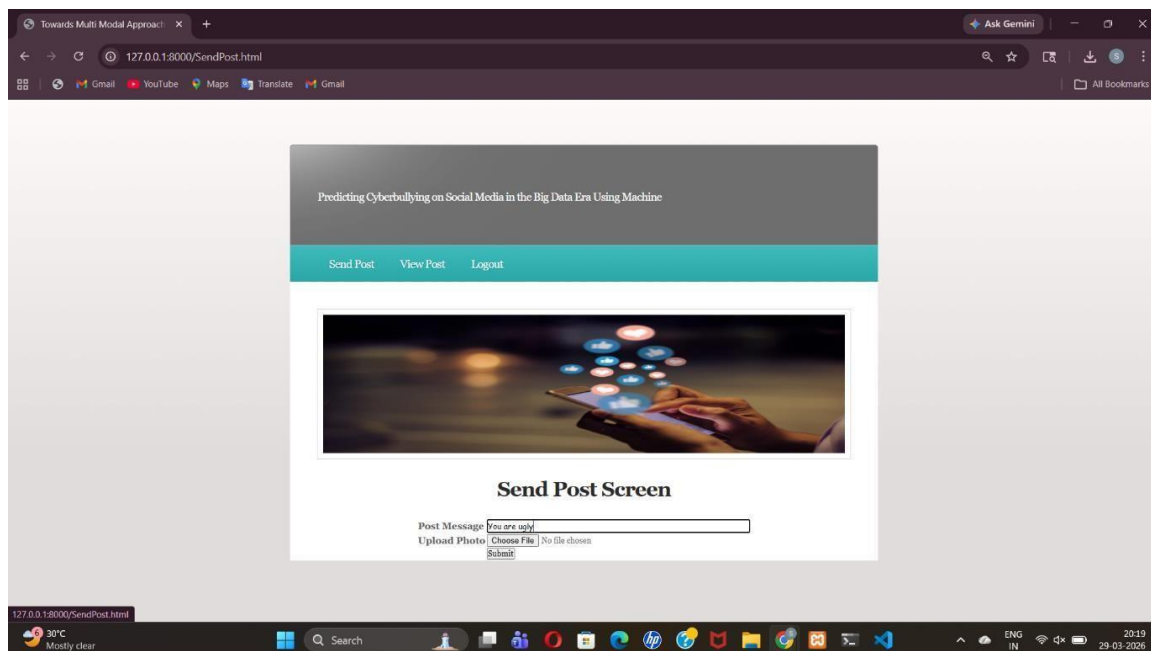


Fig: SCREENSHOT 12

As we see in the above figure, user can post anything after login. Also, user can view all his posts as we see below.

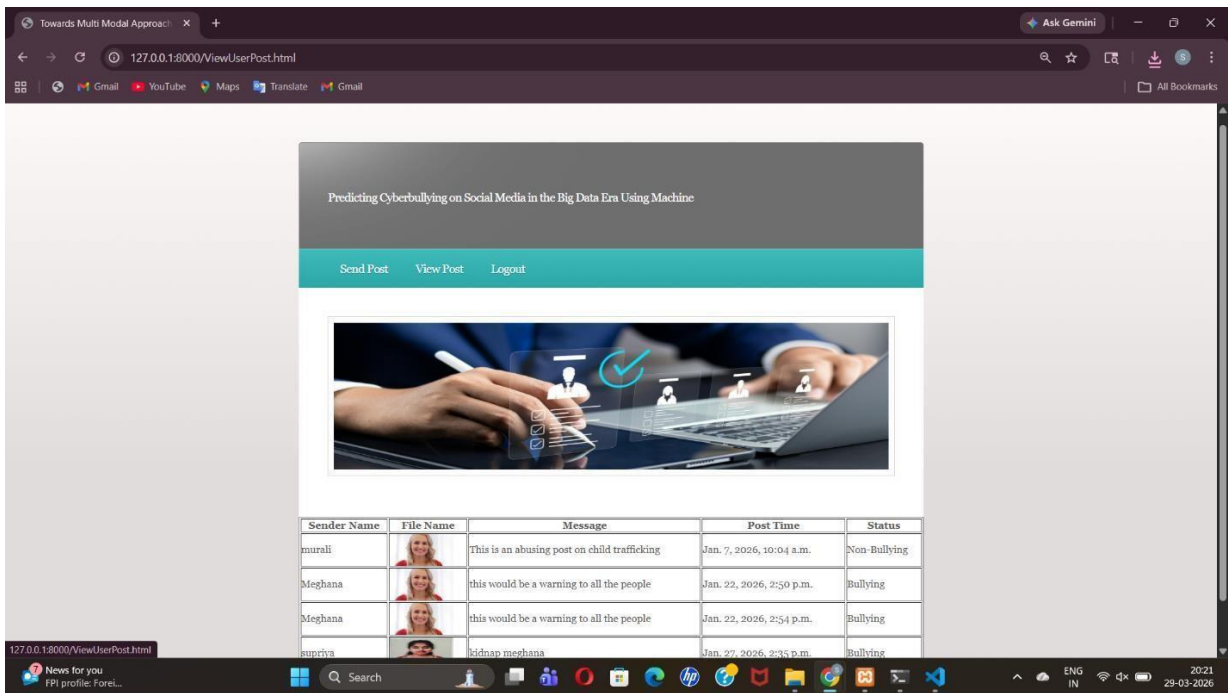


Fig: SCREENSHOT 13

There is also admin login to monitor all the posts of users. Username is admin and password is also admin, as we see below.

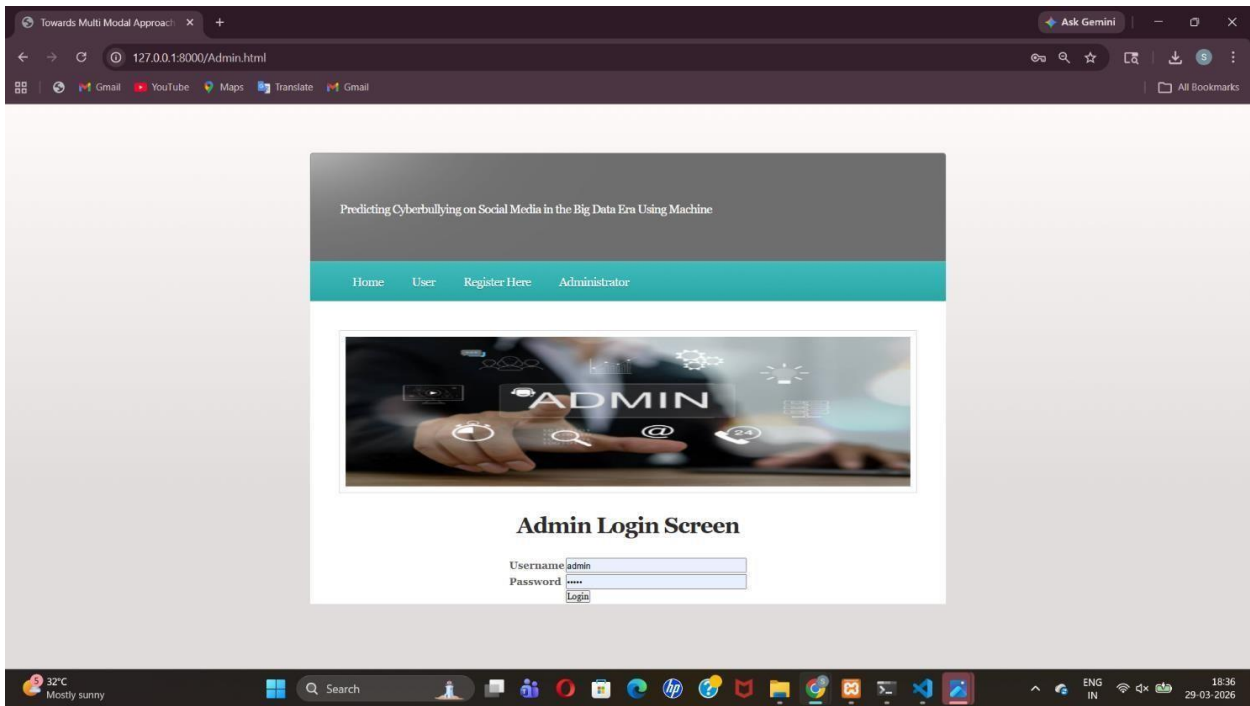


Fig: SCREENSHOT 14

After login into the admin application, we have to run any algorithm so that user can post anything, we can monitor everything.

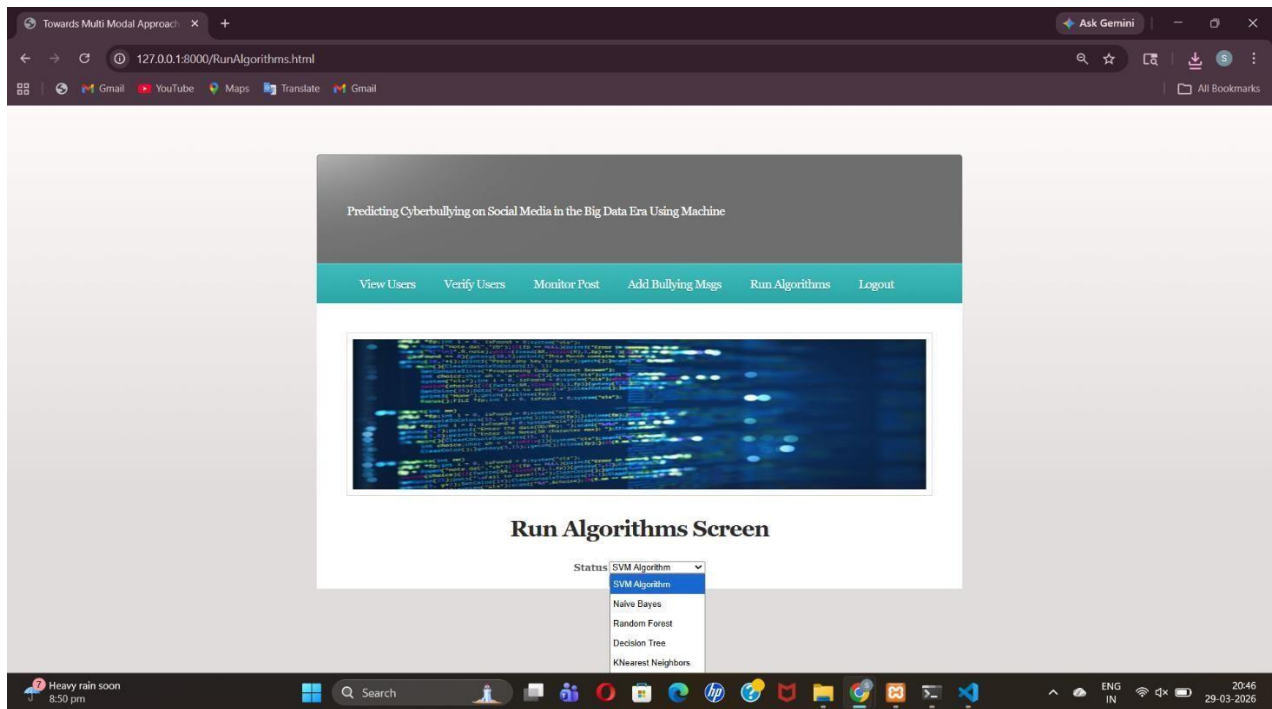


Fig: SCREENSHOT 15

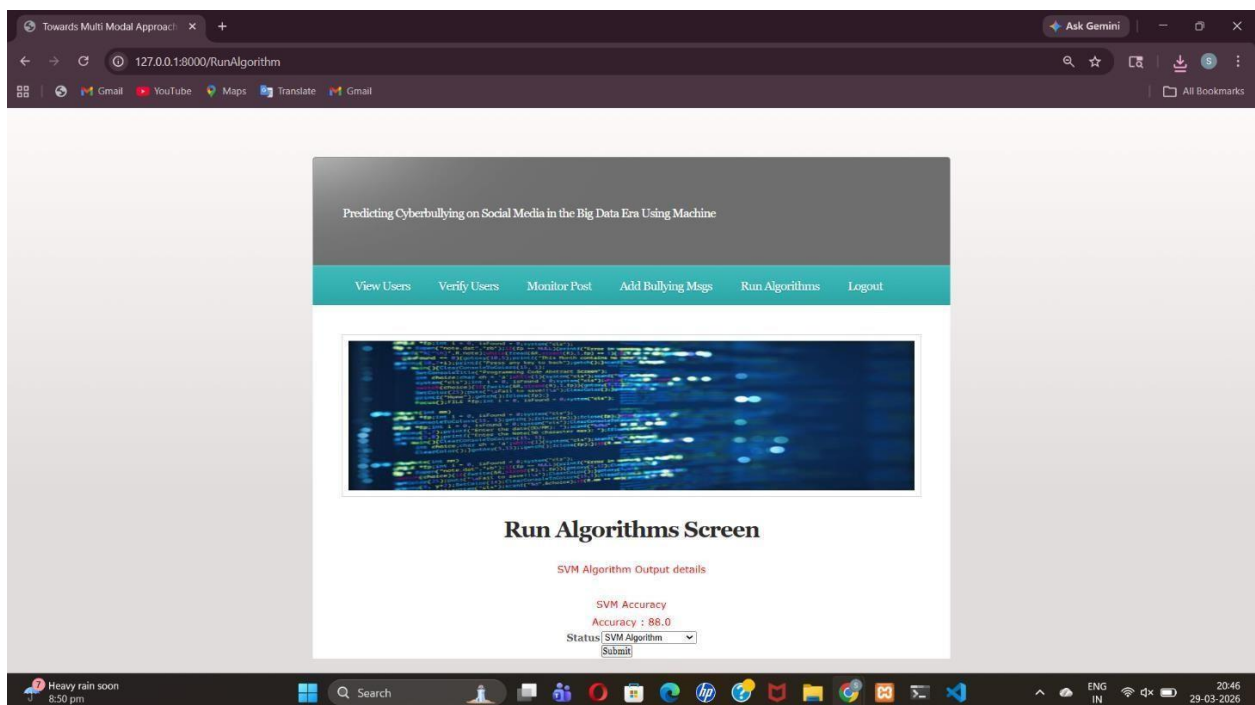


Fig: SCREENSHOT 16

So that the output will be like this after running any algorithm (as shown in the above figure). As we see below figure, we can accept or reject the users. Also view all the users i.e., we have all the data of users.

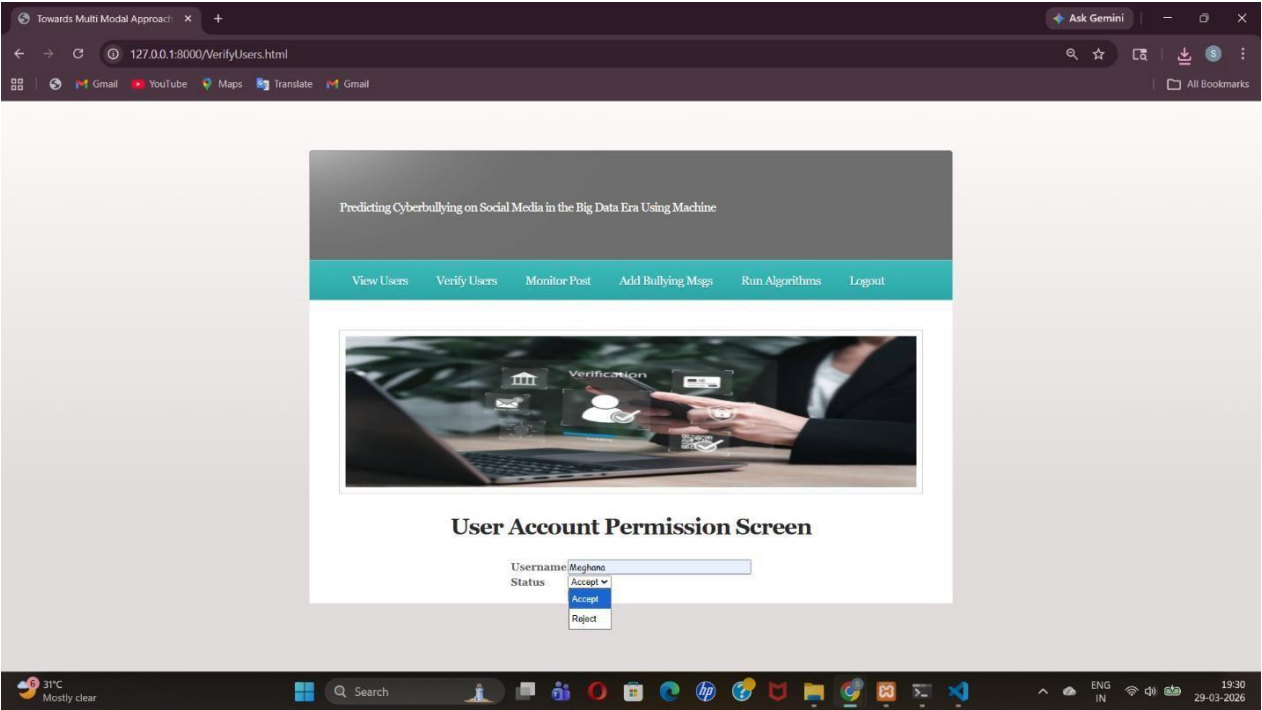


Fig: SCREENSHOT 17

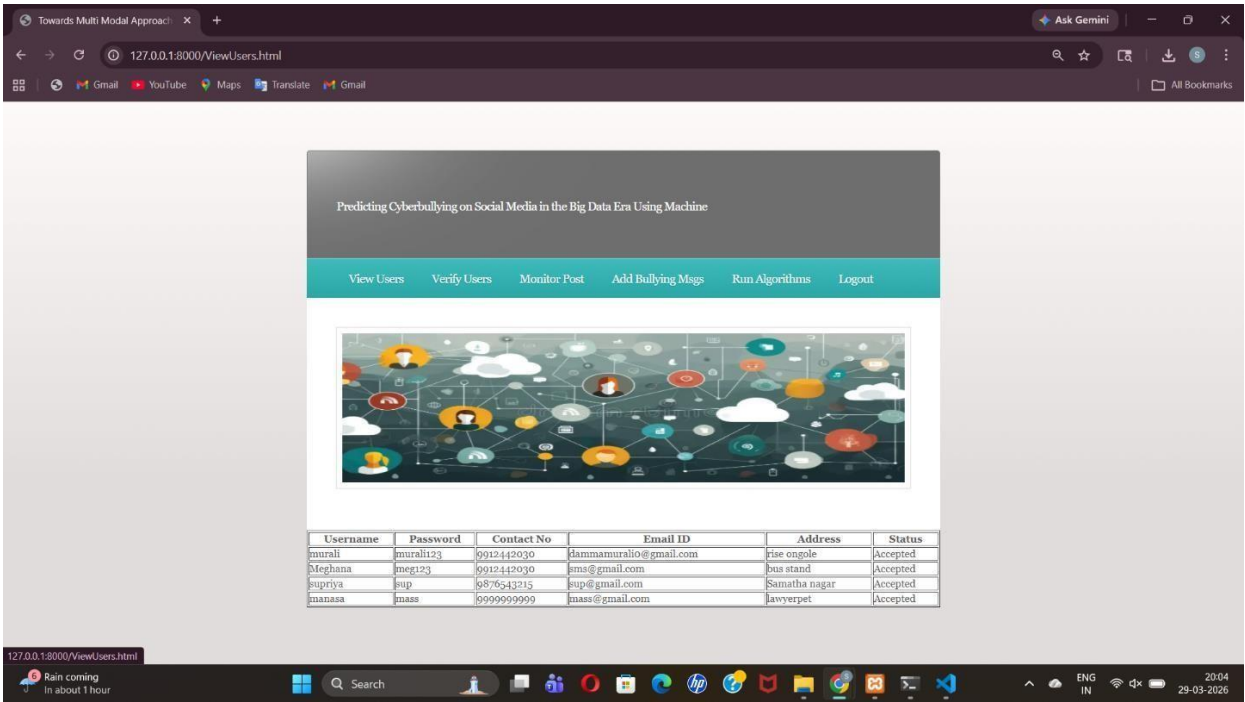


Fig: SCREENSHOT 18

The application can also monitor all the post of the users for the safety purpose, as we shown in the below figure.

In this application admin can also add messages are bullying (or) non-bullying.

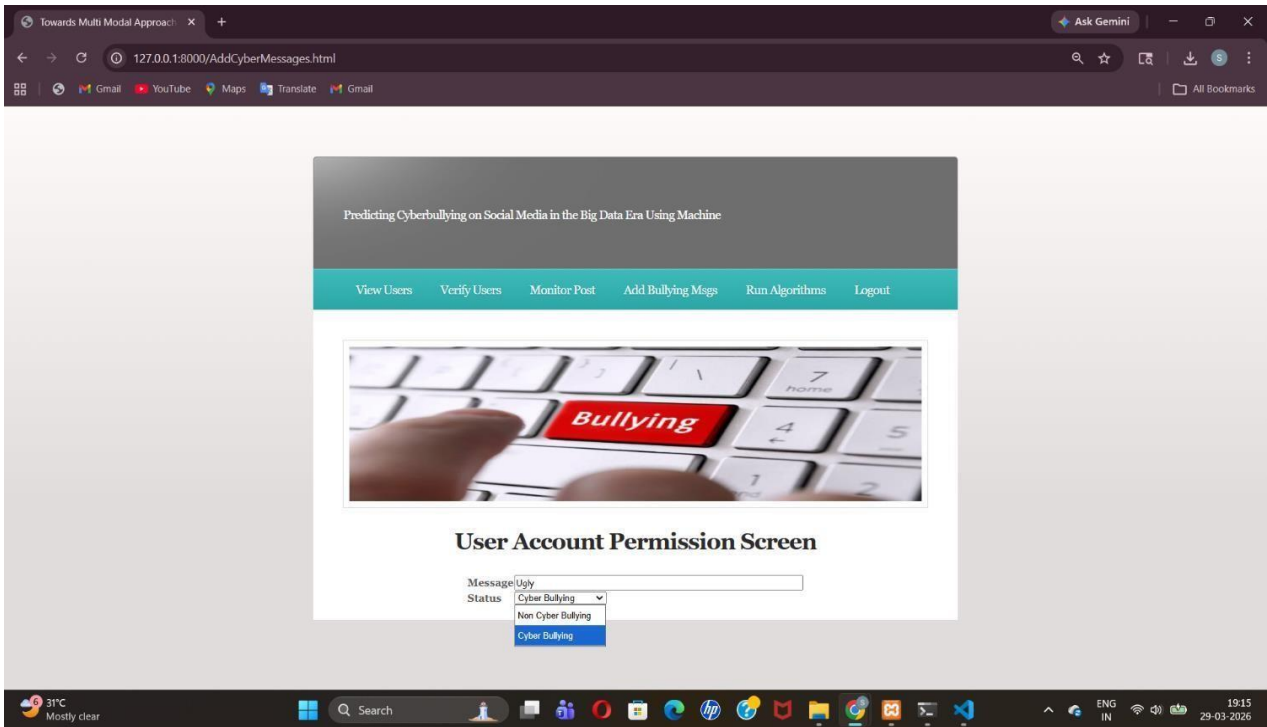


Fig: SCREENSHOT 19

If the user is new to the application, then there is an option called new registration.

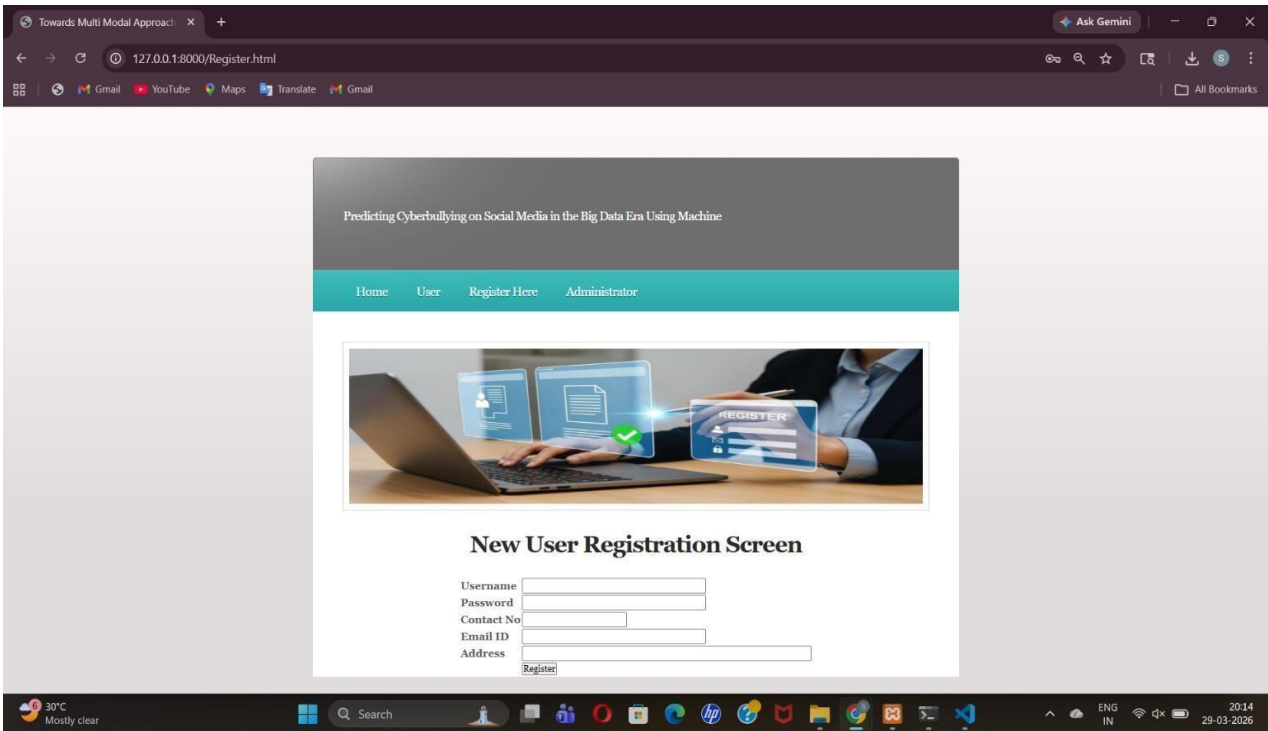


Fig: SCREENSHOT 20

CHAPTER 11

CONCLUSION & FUTURE ENHANCEMENT

11. CONCLUSION & FUTURE ENHANCEMENT

CONCLUSION

This study reviewed existing literature to detect aggressive behavior on SM websites by using machine learning approaches. I specifically reviewed four aspects of detecting cyberbullying messages by using machine learning approaches, namely, data collection, feature engineering, construction of cyberbullying detection model, and evaluation of constructed cyberbullying detection models. Several types of discriminative features that were used to detect cyberbullying in online social networking sites were also summarized. In addition, the most effective supervised machine learning classifiers for classifying cyberbullying messages in online social networking sites were identified. One of the main contributions of current paper is the definition of evaluation metrics to successfully identify the significant parameter so the various machine learning algorithms can be evaluated against each other. Most importantly i summarized and identified the important factors for detecting cyberbullying through machine learning techniques specially supervised learning. For this purpose, we have used accuracy, precision recall and f-measure which gives us the area under the curve function for modeling the behaviors in cyberbullying. Finally, the main issues and open research challenges were described and discussed.

FUTURE ENHANCEMENT

In the current implementation we do not consider the length of the tweet. As per restrictions, a tweet may not exceed 140 characters, but when we run the proposed algorithms, we do not consider whether the tweet is using the maximum available character space.

For the target high frequency words to contribute more towards the score of a Tweet its frequency should ideally be substantially more than the more common words, whose contribution should be insignificant.

Analyzing features in Twitter trend mining typically involves examining key aspects such as tweet content, engagement metrics, hashtags, and user interactions to identify trends. Natural Language Processing (NLP) techniques can be applied to understand the sentiment and topics within tweets. Additionally, tracking the frequency of hashtags and mentions can help uncover popular themes. Social network analysis may reveal influential users driving trends.

CHAPTER 12

BIBLIOGRAPHY

12. BIBLIOGRAPHY

1. V. Subrahmanian and S. Kumar, "Predicting human behavior: The next frontiers," *Science*, vol. 355, no. 6324, p. 489, 2017.
2. H. Lauw, J. C. Shafer, R. Agrawal, and A. Ntoulas, "Homophily in the digital world: A LiveJournal case study," *IEEE Internet Computing*, vol. 14, no. 2, pp. 15–23, Mar./Apr. 2010.
3. M. A. Al-Garadi, K. D. Varathan, and S. D. Ravana, "Cybercrime detection in online communications: The experimental case of cyberbullying detection in the Twitter network," *Computers in Human Behavior*, vol. 63, pp. 433–443, Oct. 2016.
4. L. Phillips, C. Dowling, K. Shaffer, N. Hodas, and S. Volkova, "Using social media to predict the future: A systematic literature review," *arXiv:1706.06134*, 2017.
5. H. Quan, J. Wu, and Y. Shi, "Online social networks and social network services: A technical survey," in *Pervasive Communication Handbook*. Boca Raton, FL, USA: CRC Press, 2011, p. 4.
6. J. K. Peterson and J. Densley, "Is social media a gang? Toward a selection, facilitation, or enhancement explanation of cyber violence," *Aggression and Violent Behavior*, 2016.
7. BBC, "Huge rise in social media," 2012. [Online]. Available: <http://www.bbc.com/news/uk-20851797>
8. P. A. Watters and N. Phair, "Detecting illicit drugs on social media using automated social media intelligence analysis (ASMIA)," in *Cyberspace Safety and Security*. Berlin, Germany: Springer, 2012, pp. 66–76.
9. M. Fire, R. Goldschmidt, and Y. Elovici, "Online social networks: Threats and solutions," *IEEE Communications Surveys & Tutorials*, vol. 16, no. 4, pp. 2019–2036, 4th Quart., 2014.
10. N. M. Shekokar and K. B. Kansara, "Security against sybil attack in social network," in *Proc. Int. Conf. Information Communication and Embedded Systems (ICICES)*, 2016, pp. 1–5.
11. J. Ratkiewicz, M. Conover, M. Meiss, B. Gonçalves, A. Flammini, and F. Menczer, "Detecting and tracking political abuse in social media," in *Proc. 5th Int. AAAI Conf. on Weblogs and Social Media*, 2011,

pp. 297–304.

12. A. Aggarwal, A. Rajadesingan, and P. Kumaraguru, “PhishAri: Automatic real time phishing detection on Twitter,” in Proc. eCrime Researchers Summit, Oct. 2012, pp. 1–12.
13. S. Yardi et al., “Detecting spam in a Twitter network,” First Monday, Jan. 2009. [Online]. Available: <https://firstmonday.org/article/view/2793/2431>
14. C. Yang, R. Harkreader, J. Zhang, S. Shin, and G. Gu, “Analyzing spammers’ social networks for fun and profit: A case study of cyber-criminal ecosystem on Twitter,” in Proc. 21st Int. Conf. on World Wide Web, 2012, pp. 71–80.
15. G. R. S. Weir, F. Toolan, and D. Smeed, “The threats of social networking: Old wine in new bottles?” Information Security Technical Report, vol. 16, no. 2, pp. 38–43, 2011.
16. M. J. Magro, “A review of social media use in e-government,” Administrative Sciences, vol. 2, no. 2, pp. 148–161, 2012.
17. M. Dadvar, D. Trieschnigg, R. Ordelman, and F. de Jong, “Improving cyberbullying detection with user context,” in Advances in Information Retrieval. Berlin, Germany: Springer, 2013, pp. 693–696.
18. Y. Chen, Y. Zhou, S. Zhu, and H. Xu, “Detecting offensive language in social media to protect adolescent online safety,” in Proc. Int. Conf. on Privacy, Security, Risk and Trust (PASSAT), Sep. 2012, pp. 71–80.
19. V. S. Chavan and S. S. Shylaja, “Machine learning approach for detection of cyber-aggressive comments by peers on social media network,” in Proc. Int. Conf. on Advances in Computing, Communications and Informatics (ICACCI), Aug. 2015, pp. 2354–2358.
20. W. Dong, S. S. Liao, Y. Xu, and X. Feng, “Leading effect of social media for financial fraud disclosure: A text mining-based analytics,” in Proc. Americas Conf. on Information Systems (AMCIS), San Diego, CA, USA, 2016.